

UNIVERSIDADE FEDERAL DE ALAGOAS

Instituto de Computacao

Framework Whitelabel para Sistemas Online de Aluguel de Casas e Apartamentos

Relatorio Tecnico de Desenvolvimento

Alunos:

Marcos Melo dos Santos

(Gateway, Reservas Core e Front-end via Figma/MCP)

Leonardo Barbosa

(Persistencia e logica interna de Auth e Catalogo)

Disciplina:

Reuso de Software e Metodologias Ageis

Periodo: 2026.1

Sumário

1	Enunciado do Problema	3
2	Planejamento Scrum por Prioridades	3
2.1	Metodologia Agil Escolhida: Scrum	3
2.2	Product Backlog	3
2.3	Sprint Backlog por Sprint	3
2.4	Justificativa das Prioridades	3
3	Tecnicas de Reuso de Software Aplicadas	5
4	Arquitetura da Solucao	5
4.1	Diagramas de Casos de Uso	6
4.2	Interfaces dos Componentes do Framework	9
4.3	Listagem dos Microservicos em Execucao	9
5	API Gateway	10
5.1	Especificacao Documentada do API Gateway	10
5.2	Microservico de Autenticacao (ms-auth)	11
5.3	Microservico de Catalogo (ms-catalogo)	11
5.4	Microservico de Reservas (ms-reservas)	12
6	Front-End Integrado	12
7	Tecnologias Utilizadas	13
7.1	Tecnologias dos Microservicos	13
7.2	Tecnologias do Framework e das Aplicacoes	13
8	Template Method e Hotspots	13
8.1	Classes Concretas e Prova de Reuso	14
8.2	Aplicacao 1: Aluguel por Temporada	15
8.3	Aplicacao 2: Aluguel de Longa Duracao	16
8.4	Como Criar uma Nova Aplicacao Reusando o Framework	16
9	Guia do Desenvolvedor	17
9.1	Pre-requisitos	17
9.2	Subindo os Servicos	17
9.3	Criando os Bancos	17
9.4	Como Estender o Framework	18

10 Funcionamento do Sistema	18
10.1 Passo a Passo das Telas	18
10.2 Exemplos de Execucao das Duas Aplicacoes	18
11 Prints do Sistema em Execucao	19
12 Repositorio GitHub	26
12.1 Observacao sobre Video	26
A Anexos:Codigo dos Microservicos	26
A.1 API Gateway	26
A.2 Microservico ms-auth	31
A.3 Microservico ms-catalogo	38
A.4 Microservico ms-reservas e Aplicacoes	45
Referencias	55

1 Enunciado do Problema

O presente trabalho tem como objetivo o desenvolvimento de um **Framework Whitelabel** para a construcao de sistemas online de aluguel de casas e apartamentos. Um produto *whitelabel* e aquele que pode ser reaproveitado e customizado por diferentes clientes (imobiliarias, plataformas de temporada, redes de coliving), mantendo um nucleo comum de regras de negocio.

O desafio central, do ponto de vista de **Reuso de Software**, e projetar uma arquitetura que maximize o reaproveitamento do fluxo de reserva, ao mesmo tempo em que permita variacoes especificas de cada modalidade de locacao. Para isso, adota-se o padrao de projeto *Template Method*, que define o esqueleto invariante do algoritmo de reserva e delega passos variaveis (*hotspots*) para subclasses.

O sistema e estruturado como um **monorepo** de microservicos em PHP 8.2 e MySQL, contemplando autenticacao, catalogo de imoveis e a engine de reservas, integrados por um *API Gateway*.

2 Planejamento Scrum por Prioridades

2.1 Metodologia Agil Escolhida: Scrum

A metodologia agil escolhida foi o **Scrum**, adaptado para uma dupla e para o calendario da disciplina. O projeto foi dividido em sprints curtas, cada uma com uma meta objetiva, um incremento entregavel e registro em branch/commit. O acompanhamento foi feito por backlog priorizado, validacao incremental e revisao do incremento ao final de cada etapa.

No contexto do trabalho, os papeis foram simplificados: a dupla atuou como time de desenvolvimento; os requisitos da disciplina funcionaram como *Product Owner*; e a revisao de cada entrega funcionou como validacao da sprint. Essa adaptacao manteve o foco principal do Scrum: entregar valor em ciclos curtos, inspecionar o que foi feito e ajustar a proxima etapa.

2.2 Product Backlog

O desenvolvimento seguiu uma abordagem incremental, organizada em um *Product Backlog* priorizado. Cada item foi tratado como uma *Sprint* curta, com entrega versionada ao final de cada etapa.

2.3 Sprint Backlog por Sprint

2.4 Justificativa das Prioridades

Os itens de infraestrutura e seguranca (Gateway e Autenticacao) recebem prioridade alta por serem dependencias transversais. A engine de reservas, sendo o nucleo de reuso e o diferencial academico do trabalho, tambem possui prioridade alta. O catalogo e as classes concretas dependem dessas bases, e o front-end e a camada de apresentacao final, com prioridade mais baixa por nao bloquear as demais entregas.

Prioridade	Item do Backlog (User Story)	Status
1 (Alta)	Inicializacao do repositorio: <code>.gitignore</code> , <code>README</code> e relatorio tecnico.	Concluido
2 (Alta)	API Gateway: roteamento central <i>stateless</i> e comunicacao interna via <code>cURL</code> .	Concluido
3 (Alta)	Microservico de Autenticacao (<code>ms-auth</code>) com JWT e banco <code>db_auth</code> .	Concluido
4 (Media)	Microservico de Catalogo (<code>ms-catalogo</code>) e banco <code>db_catalogo</code> .	Concluido
5 (Alta)	Engine de Reservas (<code>ms-reservas</code>): classe abstrata <code>RentEngine</code> (Template Method).	Concluido
6 (Media)	Classes concretas <code>LongTermRent</code> e <code>VacationRent</code> , provando o reuso.	Concluido
7 (Baixa)	Front-end integrado no Gateway (Blade/HTML) consumindo o ecossistema.	Concluido

Tabela 1: Product Backlog priorizado do framework.

Sprint	Meta	Itens do Sprint Backlog	Entrega
1	Inicializar o monorepo	Criar estrutura de pastas; adicionar <code>README</code> ; preparar relatorio em LaTeX; definir escopo e arquitetura inicial.	Base do projeto
2	Implementar Gateway	Criar <code>gateway</code> ; configurar roteamento <code>/api/*</code> ; implementar cliente HTTP via <code>cURL</code> ; servir a view principal.	Gateway funcional
3	Implementar autenticao	Criar <code>ms-auth</code> ; modelar usuarios; implementar cadastro, login, JWT, validacao de token e seed administrativo.	Auth JWT
4	Implementar catalogo	Criar <code>ms-catalogo</code> ; modelar imoveis; implementar CRUD, filtros, validacoes e vinculo logico com proprietario.	Catalogo REST
5	Implementar framework de reservas	Criar <code>ms-reservas</code> ; implementar <code>RentEngine</code> ; definir frozen spots e hotspots; criar persistencia de reservas.	Nucleo reutilizavel
6	Criar aplicacoes concretas	Implementar <code>VacationRent</code> ; implementar <code>LongTermRent</code> ; adicionar <code>CreditCheckComponent</code> ; registrar modalidades na fabrica.	Duas aplicacoes por reuso
7	Integrar interface e seguranca	Criar front-end integrado no Gateway; consumir Auth, Catalogo e Reservas; proteger rotas de escrita via validacao JWT.	Sistema integrado

Tabela 2: Sprint Backlog resumido por sprint.

3 Técnicas de Reuso de Software Aplicadas

O projeto aplica multiplas tecnicas de reuso, em diferentes niveis de granularidade:

Frameworks de Aplicacao O proprio produto e um *framework whitelabel*: fornece um esqueleto reutilizavel que e instanciado por diferentes aplicacoes de aluguel.

Padroes de Projeto Uso do *Template Method* para fixar o fluxo de reserva e variar passos especificos via *hotspots*.

Reuso por Heranca As modalidades `VacationRent` e `LongTermRent` herdam de `RentEngine`, reaproveitando o algoritmo comum.

Reuso por Composicao A funcionalidade `CreditCheckComponent` e injetada em `LongTermRent` por composicao, evitando heranca rigida e favorecendo flexibilidade.

Reuso de Servicos (SOA) A arquitetura de microservicos permite que autenticacao e catalogo sejam reutilizados por qualquer cliente do ecossistema.

Componentes de Software O `CreditCheckComponent` representa um componente plugavel do framework: ele encapsula uma regra especifica, e e usado por composicao somente pela aplicacao que precisa de analise de credito.

Assim, as tecnicas de reuso utilizadas foram: **framework de aplicacao**, **microservicos**, **componentes de software**, **Template Method**, **heranca** e **composicao**. O framework fornece o fluxo comum; os microservicos separam capacidades reutilizaveis; e os hotspots permitem adaptar o comportamento sem reescrever a engine.

4 Arquitetura da Solucao

A solucao adota uma arquitetura de **microservicos** em *monorepo*, com isolamento de dados (*Database per Service*). O cliente comunica-se exclusivamente com o Gateway, que orquestra as chamadas internas.

Servico	Porta	Banco	Responsabilidade
gateway	8003	—	Roteamento e composicao (cURL)
ms-auth	8000	db_auth	Autenticacao JWT
ms-catalogo	8001	db_catalogo	Catalogo de imoveis
ms-reservas	8002	db_reservas	Engine de reservas

Tabela 3: Mapeamento de servicos, portas e bancos de dados.

```
[ Cliente / Browser ]
  |
  v
[ API Gateway :8003 ] --- stateless, roteia via cURL
  | | |
```

```

v v v
ms-auth ms-catalogo ms-reservas
:8000 :8001 :8002
db_auth db_catalogo db_reservas

```

Listing 1: Topologia logica de comunicacao.

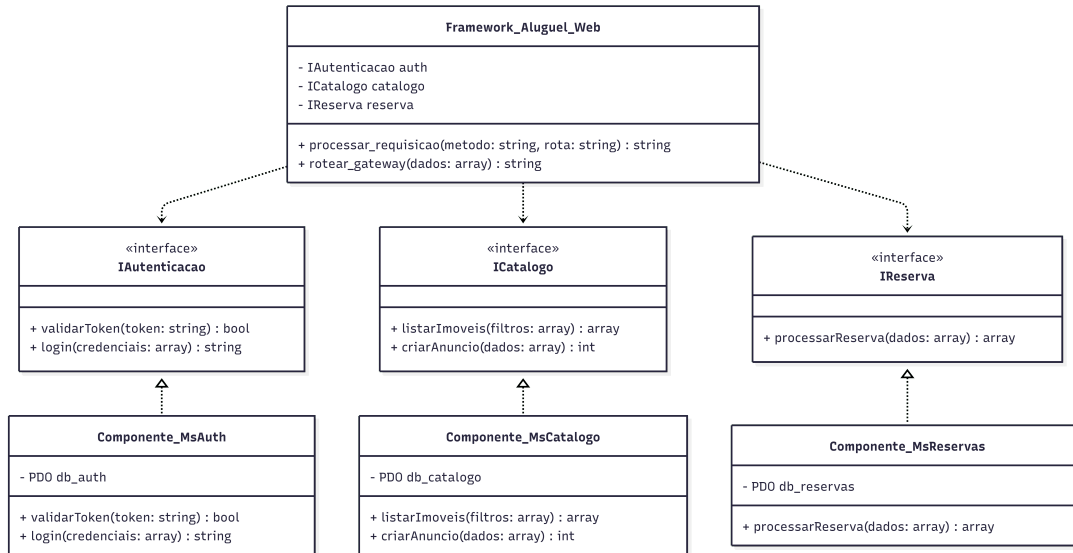


Figura 1: Diagrama de classes estrutural do framework, com a classe de framework e os componentes de software.

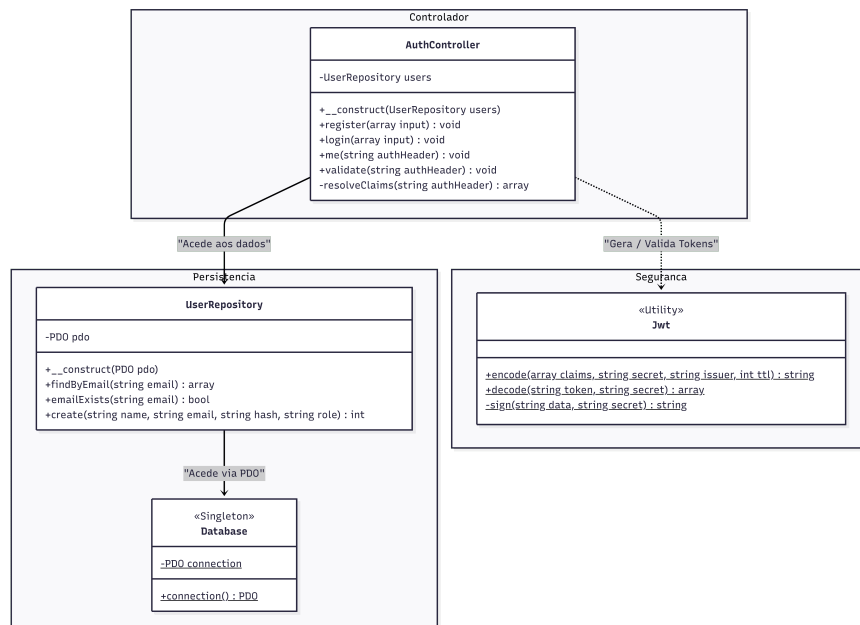


Figura 2: Diagrama de classes do componente de autenticação ms-auth.

4.1 Diagramas de Casos de Uso

Os diagramas de casos de uso mostram a visão funcional do sistema e do núcleo reutilizável do framework. No sistema completo, os atores acessam apenas o Gateway; a partir dele,

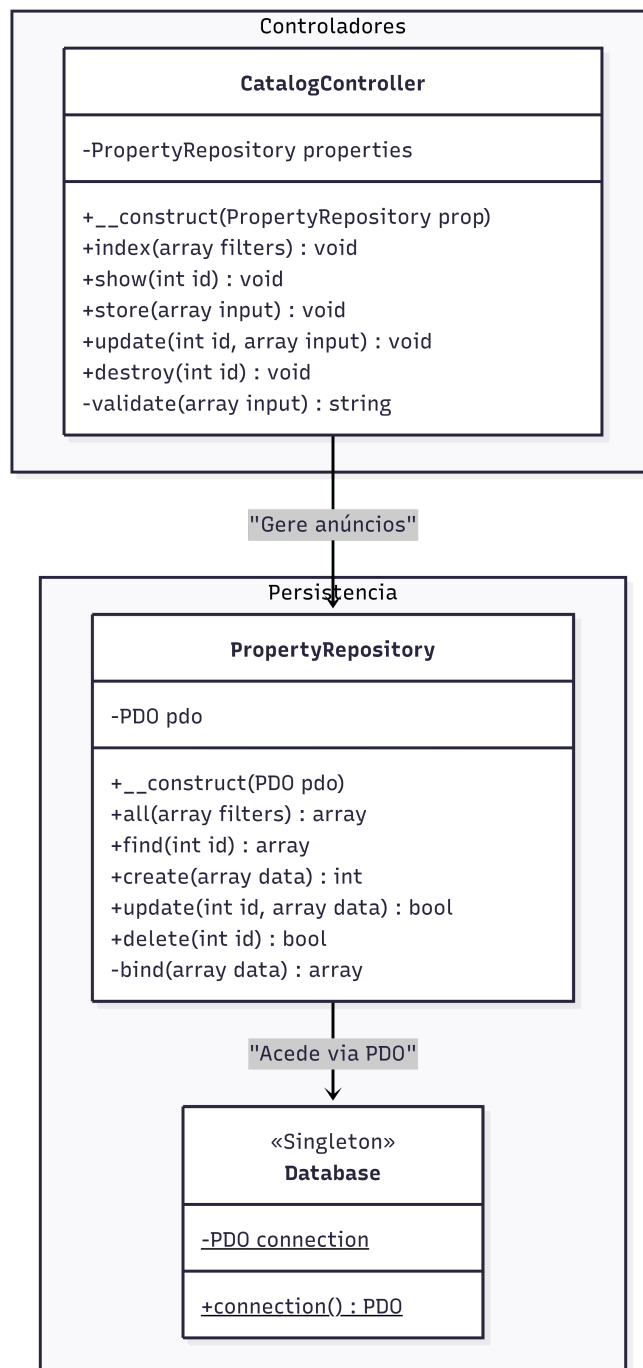


Figura 3: Diagrama de classes do componente de catalogo ms-catalogo.

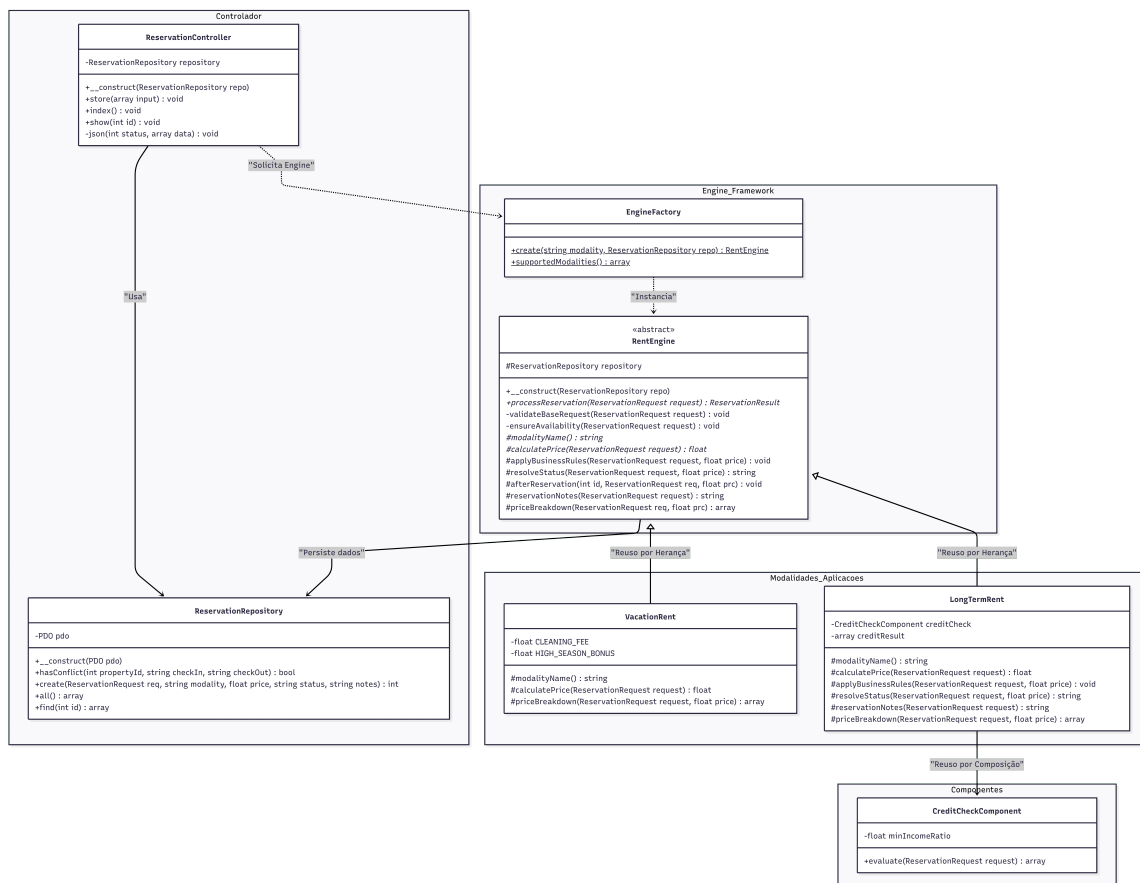


Figura 4: Diagrama de classes do componente de reservas, com Template Method, hotspots e modalidades concretas.

as funcionalidades acionam os microservicos de autenticao, catalogo e reservas.

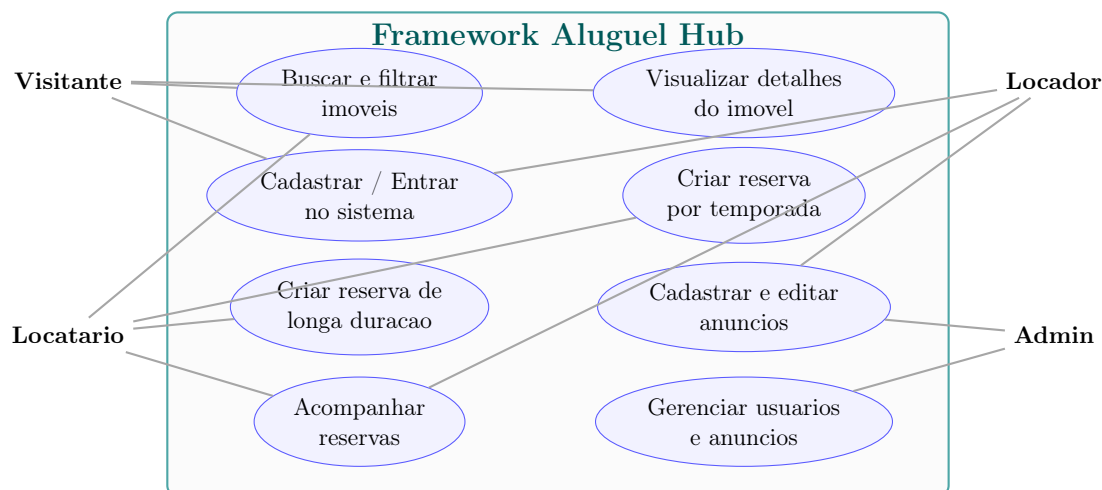


Figura 5: Diagrama de casos de uso do sistema de aluguel whitelabel.

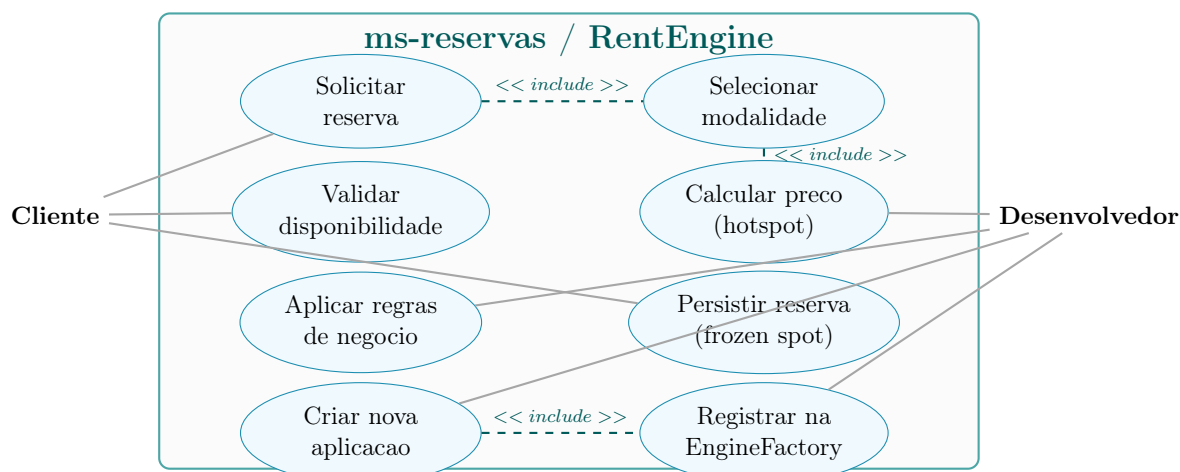


Figura 6: Diagrama de casos de uso do framework de reservas e seus hotspots.

4.2 Interfaces dos Componentes do Framework

A modelagem dos componentes considera suas interfaces de uso, isto é, os contratos publicos pelos quais cada componente é consumido dentro do framework.

4.3 Listagem dos Microservicos em Execucao

A tela principal do sistema, servida pelo Gateway em `http://localhost:8003`, destaca o uso dos microservicos por meio das rotas `/api/auth`, `/api/imoveis` e `/api/reservas`. Para evidenciar os servicos em execucao, cada microservico tambem expoe uma rota `/health`:

```
curl http://localhost:8000/health
# {"service":"ms-auth","status":"ok"}
```

Componente	Interface/Contrato	Uso no Framework
AuthController	register, login, me, validate, logout, refresh	Autenticacao, emissao e validacao de JWT para rotas protegidas.
CatalogController	index, show, store, update, destroy	Cadastro, consulta e manutencao dos imoveis disponiveis para reserva.
RentEngine	processReservation e hotspots protegidos	Classe do framework que fixa o fluxo de reserva e permite adaptacao por subclasses.
CreditCheckComponent	evaluate	Componente reutilizavel de analise de credito usado por composicao na modalidade de longa duracao.
EngineFactory	create, supportedModalities	Ponto de extensao que registra e instancia aplicacoes concretas do framework.

Tabela 4: Interfaces dos componentes usados no framework.

```

curl http://localhost:8001/health
# {"service":"ms-catalogo","status":"ok"}

curl http://localhost:8002/health
# {"service":"ms-reservas","status":"ok"}

curl http://localhost:8003
# Interface principal servida pelo API Gateway

```

Listing 2: Verificacao dos microsservicos em execucao e suas portas.

5 API Gateway

O *API Gateway* e o unico ponto de entrada do sistema (*Single Entry Point*). Suas caracteristicas principais sao:

- **Stateless:** nao mantem estado de sessao; cada requisicao carrega o token JWT necessario, repassado aos microsservicos.
- **Roteamento:** mapeia rotas externas (ex.: `/api/imoveis`) para os servicos internos correspondentes.
- **Comunicacao via cURL:** as chamadas internas sao feitas por HTTP com a extensao cURL do PHP nativo, mantendo baixo acoplamento.
- **Apresentacao:** serve as *views* (Blade/HTML) do front-end integrado.

5.1 Especificacao Documentada do API Gateway

O Gateway esta implementado em `gateway/src/Router.php`. Ele recebe toda requisicao externa, identifica o prefixo `/api/{servico}` e encaminha a chamada para o microsservico

correto usando `HttpClient`. O corpo da requisicao, o metodo HTTP e o cabeçalho `Authorization` sao preservados, mantendo a arquitetura stateless.

```
// Rotas iniciadas em /api/{servico}/... sao encaminhadas ao microservico.
// Rotas de escrita em imoveis/reservas exigem validacao JWT em ms-auth.
$isProtected = ($service === 'imoveis' && in_array($method, ['POST', 'PUT', 'DELETE'])
    ↪ )
    || ($service === 'reservas' && in_array($method, ['POST', 'PUT', 'DELETE'])
    ↪ );

if ($isProtected) {
    $authCheck = $this->http->forward('POST', $serviceMap['auth'] . '/validate', '',
    ↪ $headers);
    if ($authCheck['status'] !== 200) {
        $this->json(401, ['error' => 'Nao autorizado pelo Gateway.']);
        return;
    }
}
```

Listing 3: Trecho documentado do roteamento central do Gateway.

5.2 Microservico de Autenticacao (ms-auth)

O `ms-auth` (porta 8000, banco `db_auth`) e responsavel pela identidade do ecossistema. Implementa JWT no algoritmo HS256 de forma autocontida, expondo os endpoints `/register`, `/login`, `/me` e `/validate`. As senhas sao armazenadas com `bcrypt` e o acesso a dados usa PDO com *prepared statements*. Por ser stateless, o token viaja em cada requisicao (cabeçalho `Authorization`), sendo repassado pelo Gateway aos demais servicos.

Especificacao da API do ms-auth:

- POST `/api/auth/register`: cadastra usuario comum com papel locador ou locatario.
- POST `/api/auth/login`: autentica email/senha e retorna JWT.
- GET `/api/auth/me`: retorna dados do usuario autenticado.
- POST `/api/auth/validate`: valida token para o Gateway.
- POST `/api/auth/logout`: revoga token.
- POST `/api/auth/refresh`: emite novo token.

5.3 Microservico de Catalogo (ms-catalogo)

O `ms-catalogo` (porta 8001, banco `db_catalogo`) gerencia o cadastro de imoveis (casas e apartamentos) com uma API REST de CRUD: GET `/` (com filtros por cidade, tipo e disponibilidade), GET `/id`, POST `/`, PUT `/id` e DELETE `/id`. O relacionamento com o usuario proprietario (`owner_id`) e logico, sem chave estrangeira fisica, respeitando o isolamento do padrao *Database per Service*.

Especificacao da API do ms-catalogo:

- GET `/api/imoveis`: lista imoveis com filtros por cidade, tipo, disponibilidade, paginação e ordenação.
- GET `/api/imoveis/{id}`: detalha um imóvel.
- POST `/api/imoveis`: cria anúncio. Rota protegida para admin ou locador.
- PUT `/api/imoveis/{id}`: atualiza anúncio. Exige dono ou administrador.
- DELETE `/api/imoveis/{id}`: remove anúncio. Exige dono ou administrador.

5.4 Microserviço de Reservas (ms-reservas)

O `ms-reservas` (porta 8002, banco `db_reservas`) contém a classe `RentEngine`, que é a classe de framework. A API recebe uma reserva e seleciona a aplicação concreta pelo campo `modality`. As modalidades disponíveis são `vacation` e `long_term`.

Especificação da API do `ms-reservas`:

- GET `/api/reservas`: lista reservas e informa modalidades suportadas.
- GET `/api/reservas/{id}`: detalha uma reserva.
- POST `/api/reservas`: cria reserva usando a engine escolhida por `modality`. Rota protegida por token JWT no Gateway.

6 Front-End Integrado

A camada de apresentação é servida pelo próprio Gateway (porta 8003). O design foi derivado via **Figma MCP** (`get_figma_data`), mesclando dois kits da Figma Community com paleta de cores própria (identidade UFAL):

- **M-Rent** (Property Management): cards de imóvel com *border-radius* 12px, badges de tipo e estrutura de listagem com filtros.
- **Estatery** (Real Estate SaaS UI Kit): *hero*, *Search Bar*, abas por tipo de imóvel e barra de resultados.

A view consome exclusivamente as rotas `/api/*`:

- **Autenticacao**: login via POST `/api/auth/login`; token JWT no `localStorage`, reenviado em `Authorization`.
- **Catalogo**: busca com filtros de API (`city`, `type`, `available`) e filtros client-side (`preco`, `quartos`, `area`, `ordenacao`).
- **Reservas**: criação via POST `/api/reservas` com modalidades `vacation` e `long_term`.

Assets em `gateway/public/assets/css/app.css` e `gateway/views/home.php`.

7 Tecnologias Utilizadas

7.1 Tecnologias dos Microservicos

Servico	Linguagem	Banco	Tecnologias
gateway	PHP 8.2	–	Servidor embutido do PHP, cURL, HTML, CSS e JavaScript.
ms-auth	PHP 8.2	MySQL db_auth	PDO, JWT HS256, bcrypt, Composer e variaveis .env.
ms-catalogo	PHP 8.2	MySQL db_catalogo	PDO, API REST, filtros, paginacao, autorizacao por cabecalhos validados.
ms-reservas	PHP 8.2	MySQL db_reservas	Template Method, classes abstratas, heranca, composicao e Factory.

Tabela 5: Tecnologias por microservico.

7.2 Tecnologias do Framework e das Aplicacoes

O framework foi implementado com PHP orientado a objetos, usando namespaces, `strict_types`, classes `final`, classe abstrata, DTOs e repositorios. As aplicacoes construidas por reuso sao as modalidades `VacationRent` e `LongTermRent`. Ambas reutilizam a mesma engine, a mesma API de reservas, a mesma persistencia e a mesma integracao pelo Gateway.

No front-end, a interface usa HTML, CSS e JavaScript servidos pelo Gateway. A interface principal permite autenticar o usuario, listar imoveis, aplicar filtros e criar reservas, evidenciando o uso integrado dos microservicos.

8 Template Method e Hotspots

O nucleo de reuso e a classe abstrata `RentEngine`. Ela implementa o **Template Method** `processReservation()`, que fixa a sequencia invariante de passos do processo de reserva. Os **hotspots** (metodos abstratos ou *hooks*) representam os pontos de variacao implementados por cada subclasse.

Frozen Spots Passos fixos: validacao de disponibilidade, persistencia da reserva e confirmacao. Sao implementados na classe abstrata e nao mudam.

Hot Spots Passos variaveis: calculo de preco, regras de validacao especificas e etapas opcionais (ex.: analise de credito). Sao abstratos ou *hooks*, definidos pelas subclases.

```
abstract class RentEngine
{
    // TEMPLATE METHOD: esqueleto invariante, final (nao sobrescrevivel)
```

```

final public function processReservation(ReservationRequest $req):
↳ ReservationResult
{
    $this->validateBaseRequest($req); // (1) FROZEN
    $this->ensureAvailability($req); // (2) FROZEN
    $price = $this->calculatePrice($req); // (3) HOTSPOT (abstrato)
    $this->applyBusinessRules($req, $price); // (4) HOTSPOT (hook)
    $status = $this->resolveStatus($req, $price); // (5) HOTSPOT (hook)
    $id = $this->repository->create( // (6) FROZEN: persistencia
        $req, $this->modalityName(), $price, $status, $this->reservationNotes($req)
    );
    $this->afterReservation($id, $req, $price); // (7) HOTSPOT (hook)
    return new ReservationResult(/* ... */);
}

// HOTSPOTS obrigatorios (abstratos)
abstract protected function modalityName(): string;
abstract protected function calculatePrice(ReservationRequest $req): float;

// HOTSPOTS opcionais (hooks com comportamento default)
protected function applyBusinessRules(ReservationRequest $req, float $price): void
↳ {}
protected function resolveStatus(ReservationRequest $req, float $price): string {
↳ return 'confirmed'; }
protected function afterReservation(int $id, ReservationRequest $req, float $price)
↳ : void {}
}

```

Listing 4: Template Method implementado em ms-reservas/src/Engine/RentEngine.php.

A selecao da modalidade concreta ocorre em **EngineFactory**, ponto unico de extensao do framework: registrar uma nova modalidade nao exige alterar o nucleo, respeitando o *Open/Closed Principle*.

8.1 Classes Concretas e Prova de Reuso

Duas modalidades concretas instanciam o framework, reaproveitando o mesmo Template Method e variando apenas os hotspots:

VacationRent (temporada) Reuso por **heranca**: sobrescreve somente os hotspots obrigatorios (**modalityName** e **calculatePrice**), aplicando adicional de alta temporada para estadias curtas e taxa de limpeza. Os hooks opcionais usam o comportamento default da **RentEngine**.

LongTermRent (longa duracao) Combina **heranca** e **composicao**: calcula o preco mensal com desconto para contratos longos e injeta o **CreditCheckComponent** no hook **applyBusinessRules**. O resultado da analise de credito governa o hook **resolveStatus** (**confirmed** se aprovado, **pending** caso contrario).

A Tabela 6 resume como cada modalidade exercita os pontos do framework.

```

final class LongTermRent extends RentEngine
{
    public function __construct(

```

Ponto do framework	VacationRent	LongTermRent
calculatePrice (abstract)	sobrescrito (diaria)	sobrescrito (mensal)
applyBusinessRules (hook)	default (vazio)	credit check (composicao)
resolveStatus (hook)	default (confirmed)	dinamico (credito)
Esqueleto (frozen spots)	reutilizado	reutilizado

Tabela 6: Reuso dos pontos do framework por modalidade.

```

    ReservationRepository $repository,
    private readonly CreditCheckComponent $creditCheck = new CreditCheckComponent()
) { parent::__construct($repository); }

protected function applyBusinessRules(ReservationRequest $req, float $price): void
↳ {
    $this->creditResult = $this->creditCheck->evaluate($req); // COMPOSICAO
}
protected function resolveStatus(ReservationRequest $req, float $price): string {
    return $this->creditResult['approved'] ? 'confirmed' : 'pending';
}
}

```

Listing 5: Reuso por composicao: credit check no hook do LongTermRent.

Na Etapa 6, *VacationRent* sobrescreve *calculatePrice()* com tarifas de temporada, enquanto *LongTermRent* adiciona, por composicao, o *CreditCheckComponent* dentro do *hook* *applyBusinessRules()*, demonstrando a combinacao de reuso por heranca e por composicao.

8.2 Aplicacao 1: Aluguel por Temporada

A primeira aplicacao concreta e *VacationRent*. Ela representa plataformas de aluguel por temporada, como imoveis para poucos dias. A aplicacao reutiliza os frozen spots da *RentEngine* e implementa o hotspot *calculatePrice()* com diarias, taxa de limpeza e adicional para estadias curtas.

```

final class VacationRent extends RentEngine
{
    protected function modalityName(): string
    {
        return 'vacation';
    }

    protected function calculatePrice(ReservationRequest $request): float
    {
        $nights = $request->nights();
        $subtotal = $nights * $request->dailyRate;

        if ($nights <= 3) {
            $subtotal *= 1.15;
        }

        return $subtotal + 120.0;
    }
}

```



```
}
}
```

Listing 6: Hotspots implementados na aplicacao VacationRent.

8.3 Aplicacao 2: Aluguel de Longa Duracao

A segunda aplicacao concreta e `LongTermRent`. Ela representa contratos residenciais longos. Alem de implementar o preco mensal, usa composicao com `CreditCheckComponent` para adaptar o processo pelo hotspot `applyBusinessRules()`.

```
final class LongTermRent extends RentEngine
{
    protected function modalityName(): string
    {
        return 'long_term';
    }

    protected function calculatePrice(ReservationRequest $request): float
    {
        $months = $request->months();
        $subtotal = $months * $request->monthlyRate;
        return $months >= 12 ? $subtotal * 0.90 : $subtotal;
    }

    protected function applyBusinessRules(ReservationRequest $request, float $price):
    ↪ void
    {
        $this->creditResult = $this->creditCheck->evaluate($request);
    }

    protected function resolveStatus(ReservationRequest $request, float $price): string
    {
        return ($this->creditResult['approved'] ?? false) ? 'confirmed' : 'pending';
    }
}
```

Listing 7: Hotspots implementados na aplicacao LongTermRent.

8.4 Como Criar uma Nova Aplicacao Reusando o Framework

Para criar uma nova aplicacao, por exemplo aluguel corporativo, o desenvolvedor deve:

1. Criar uma classe concreta em `ms-reservas/src/Modalities`.
2. Herdar de `RentEngine`.
3. Implementar os hotspots obrigatorios `modalityName()` e `calculatePrice()`.
4. Sobrescrever hooks opcionais se a regra exigir validacao adicional, status diferente, observacoes ou pos-processamento.
5. Registrar a nova modalidade em `EngineFactory`.

```
final class CorporateRent extends RentEngine
{
    protected function modalityName(): string
    {
        return 'corporate';
    }

    protected function calculatePrice(ReservationRequest $request): float
    {
        $months = $request->months();
        $base = $months * $request->monthlyRate;
        return $base + 500.0; // taxa de administracao corporativa
    }

    protected function applyBusinessRules(ReservationRequest $request, float $price):
    ↪ void
    {
        // Hotspot opcional: validar CNPJ, contrato empresarial ou limite de ocupacao.
    }
}

// Registro na fabrica:
return match ($modality) {
    'vacation' => new VacationRent($repository),
    'long_term' => new LongTermRent($repository),
    'corporate' => new CorporateRent($repository),
};
```

Listing 8: Exemplo de nova aplicacao por reuso do framework.

O restante do processo permanece reaproveitado: validacao base, disponibilidade, persistencia, retorno padronizado e exposicao via POST /api/reservas.

9 Guia do Desenvolvedor

9.1 Pre-requisitos

PHP 8.2+ (extensoes pdo_mysql, curl, json), MySQL 8.0+ e Composer 2.x.

9.2 Subindo os Servicos

Em terminais separados:

```
cd ms-auth && composer install && php -S localhost:8000 -t public
cd ms-catalogo && composer install && php -S localhost:8001 -t public
cd ms-reservas && composer install && php -S localhost:8002 -t public
cd gateway && composer install && php -S localhost:8003 -t public
```

9.3 Criando os Bancos

```
mysql -u root -p < ms-auth/sql/db_auth.sql
mysql -u root -p < ms-catalogo/sql/db_catalogo.sql
mysql -u root -p < ms-reservas/sql/db_reservas.sql
```

9.4 Como Estender o Framework

Para criar uma nova modalidade de aluguel, basta criar uma subclasse de `RentEngine` e implementar os *hotspots*, sem alterar o fluxo principal, respeitando o *Open/Closed Principle*.

10 Funcionamento do Sistema

10.1 Passo a Passo das Telas

O funcionamento do sistema ocorre a partir da interface principal em <http://localhost:8003>. A tela inicial é servida pelo Gateway e consome os microserviços por rotas `/api/*`.

1. **Tela inicial e busca:** o usuário visualiza a listagem de imóveis e aplica filtros por cidade, tipo, preço, quartos, área e disponibilidade. A tela usa o `ms-catalogo` via `GET /api/imoveis`.
2. **Login:** o usuário informa e-mail e senha. O Gateway envia `POST /api/auth/login` ao `ms-auth`; o token retornado fica no `localStorage`.
3. **Cadastro/edição de imóvel:** usuários `locador` ou `admin` enviam dados do anúncio. O Gateway valida o token no `ms-auth` e encaminha ao `ms-catalogo`.
4. **Reserva por temporada:** a tela envia `modality: vacation`. O `ms-reservas` aciona `VacationRent`, calculando diárias, taxa de limpeza e adicional de alta temporada.
5. **Reserva de longa duração:** a tela envia `modality: long_term`. O `ms-reservas` aciona `LongTermRent`, calcula mensalidades e chama o componente de análise de crédito.

10.2 Exemplos de Execução das Duas Aplicações

```
curl -X POST http://localhost:8003/api/reservas \
-H "Content-Type: application/json" \
-H "Authorization: Bearer TOKEN" \
-d '{
  "modality": "vacation",
  "property_id": 1,
  "check_in": "2026-07-10",
  "check_out": "2026-07-13",
  "daily_rate": 350,
  "guests": 2
}'
```

Listing 9: Criacao de reserva por temporada.

```
curl -X POST http://localhost:8003/api/reservas \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer TOKEN" \  
-d '{  
  "modality": "long_term",  
  "property_id": 1,  
  "check_in": "2026-08-01",  
  "check_out": "2027-08-01",  
  "monthly_rate": 4000,  
  "guests": 2,  
  "extras": { "monthly_income": 13000 }  
}'
```

Listing 10: Criacao de reserva de longa duracao.

Esses dois exemplos demonstram que as aplicacoes mudam apenas os hotspots, enquanto o fluxo principal permanece o mesmo.

11 Prints do Sistema em Execucao

As figuras a seguir foram capturadas com os quatro servicos em execucao (`ms-auth`, `ms-catalogo`, `ms-reservas` e `gateway`) e com os bancos provisionados pelos scripts SQL do projeto.

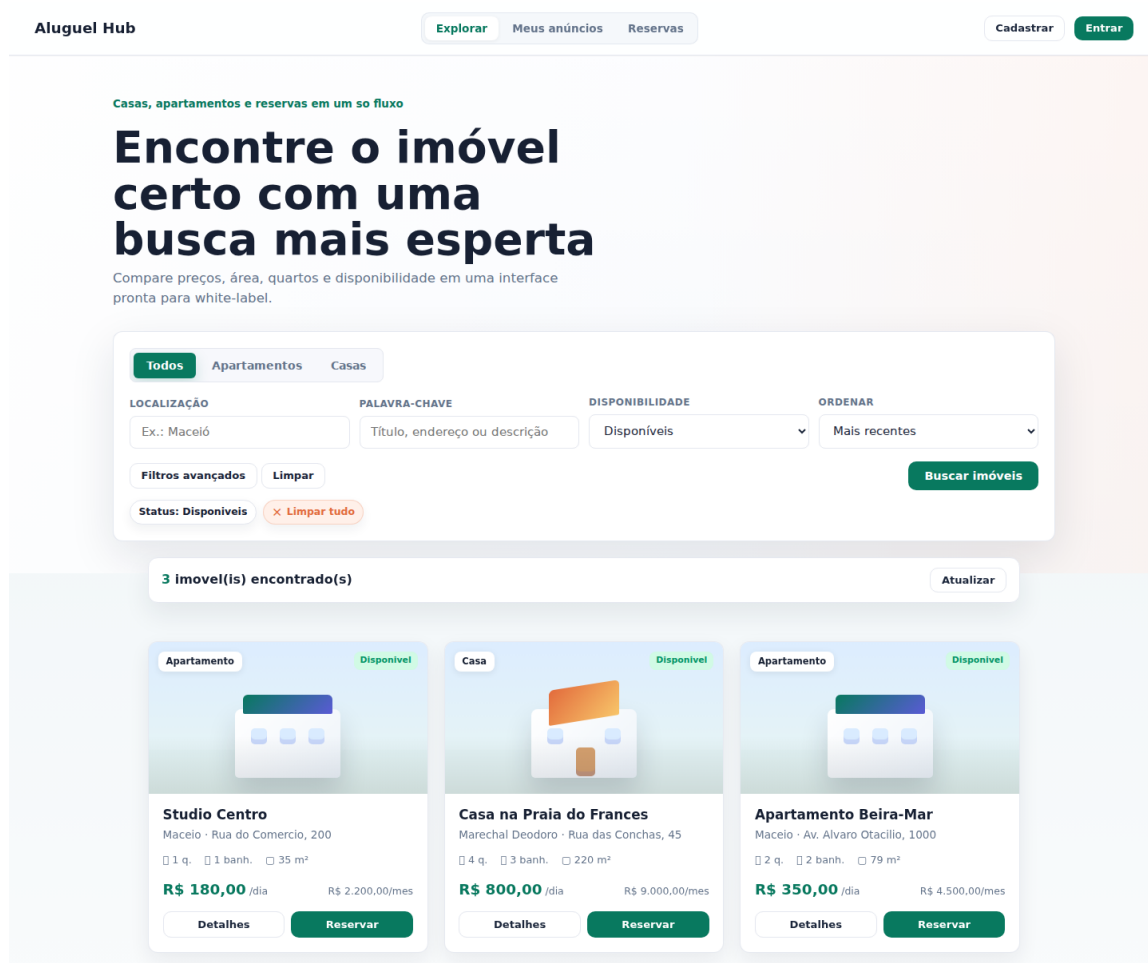


Figura 7: Tela inicial de exploração e busca de imóveis.

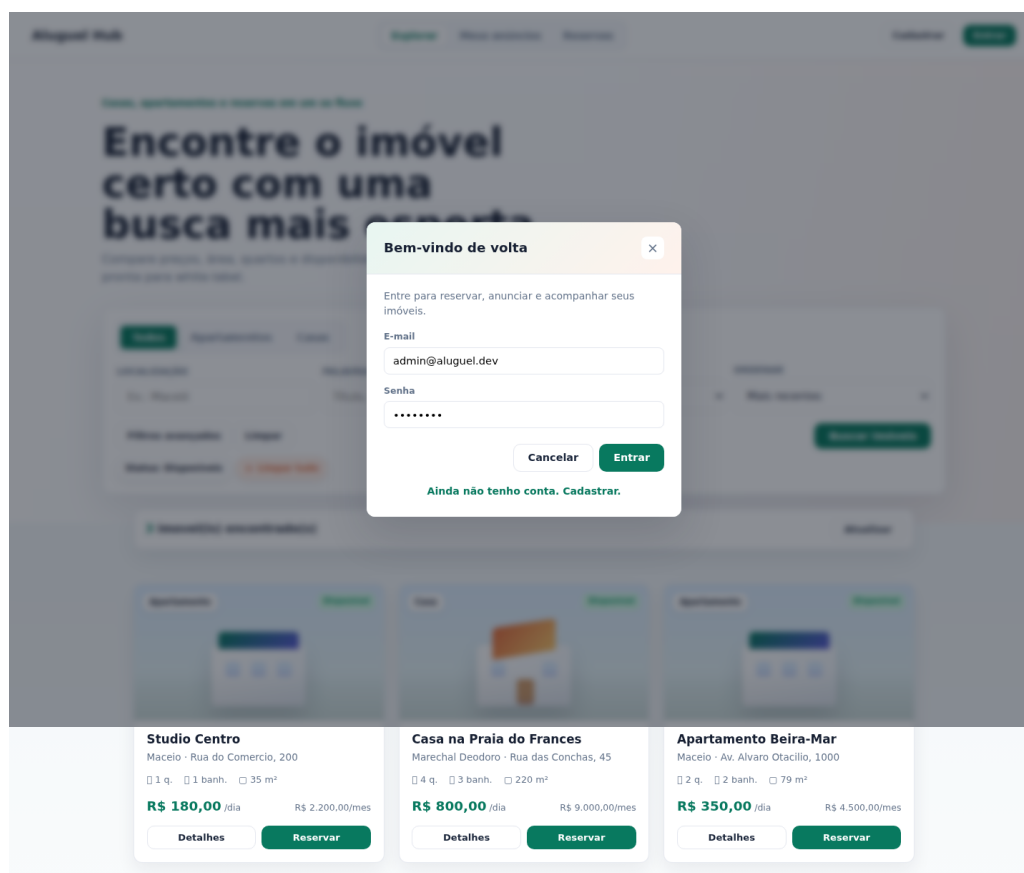


Figura 8: Modal de login consumindo o microservico de autenticao.

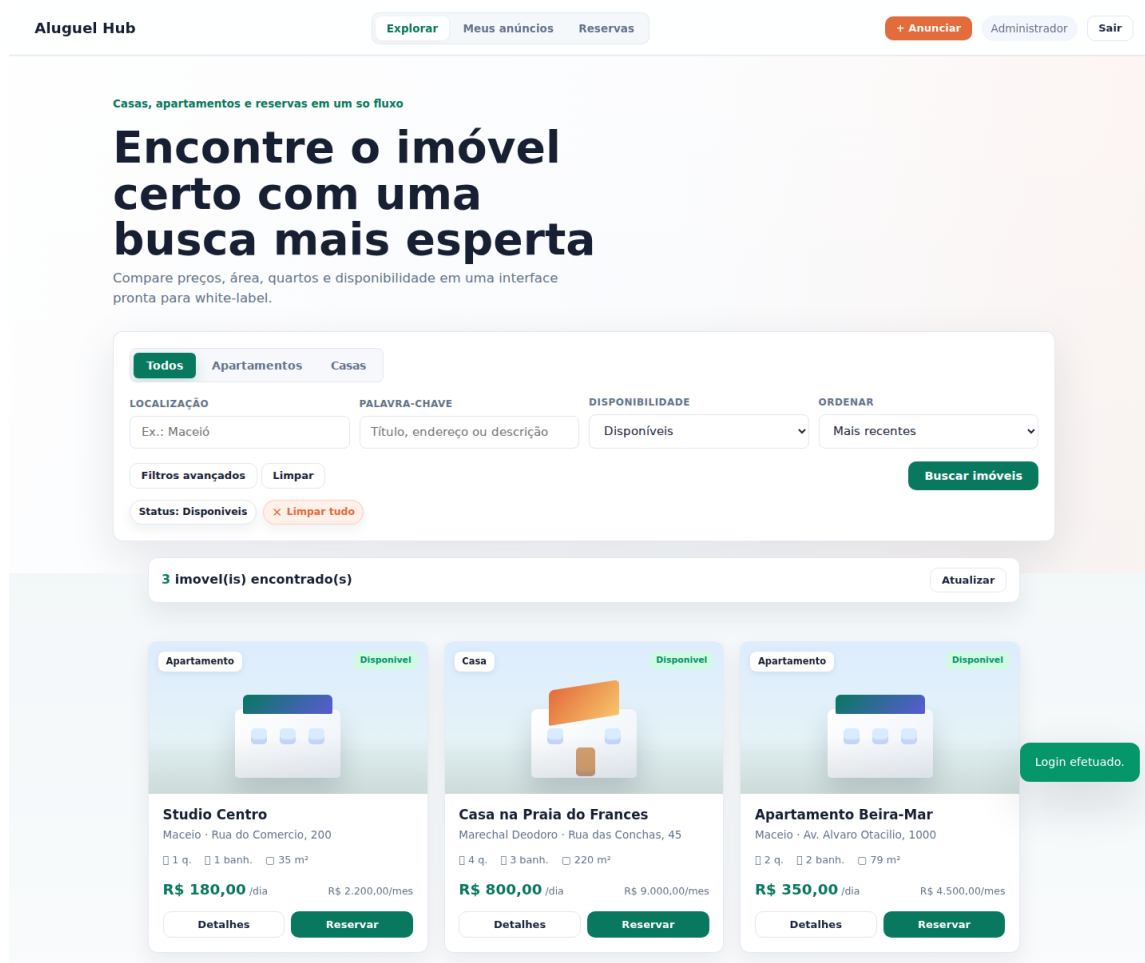


Figura 9: Tela principal após autenticação do usuário administrador.

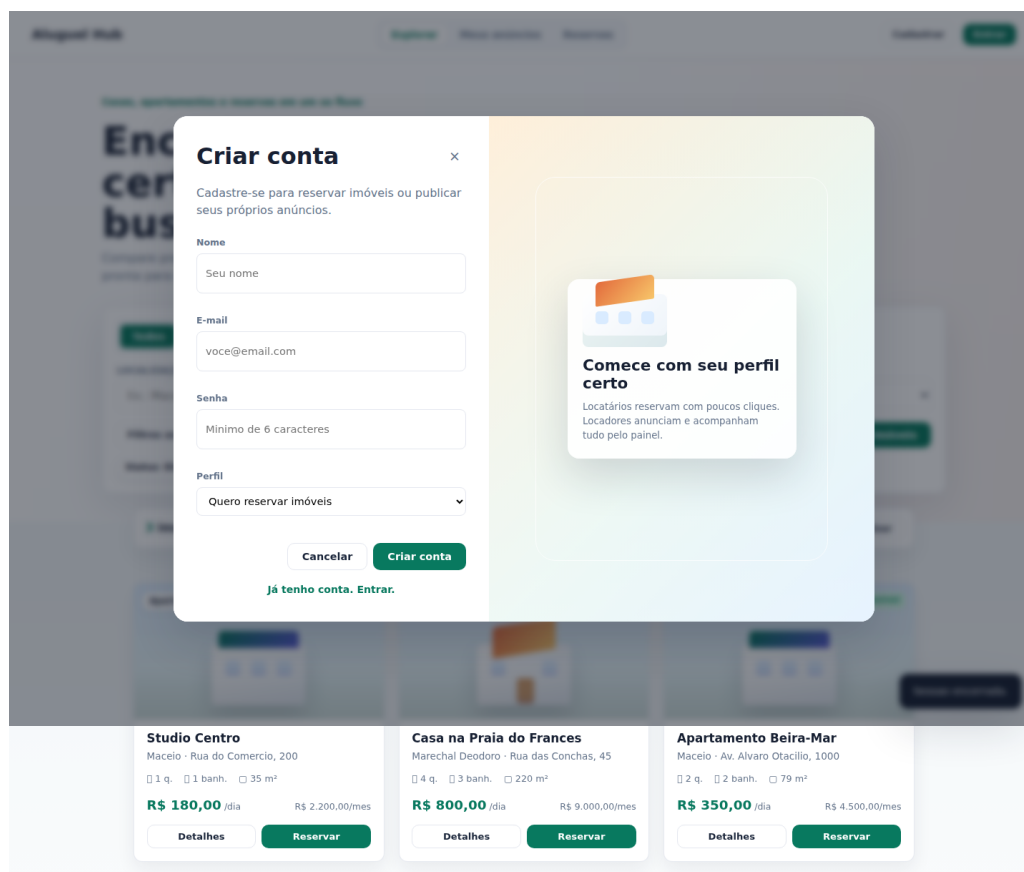


Figura 10: Modal de cadastro de usuário com escolha de perfil.

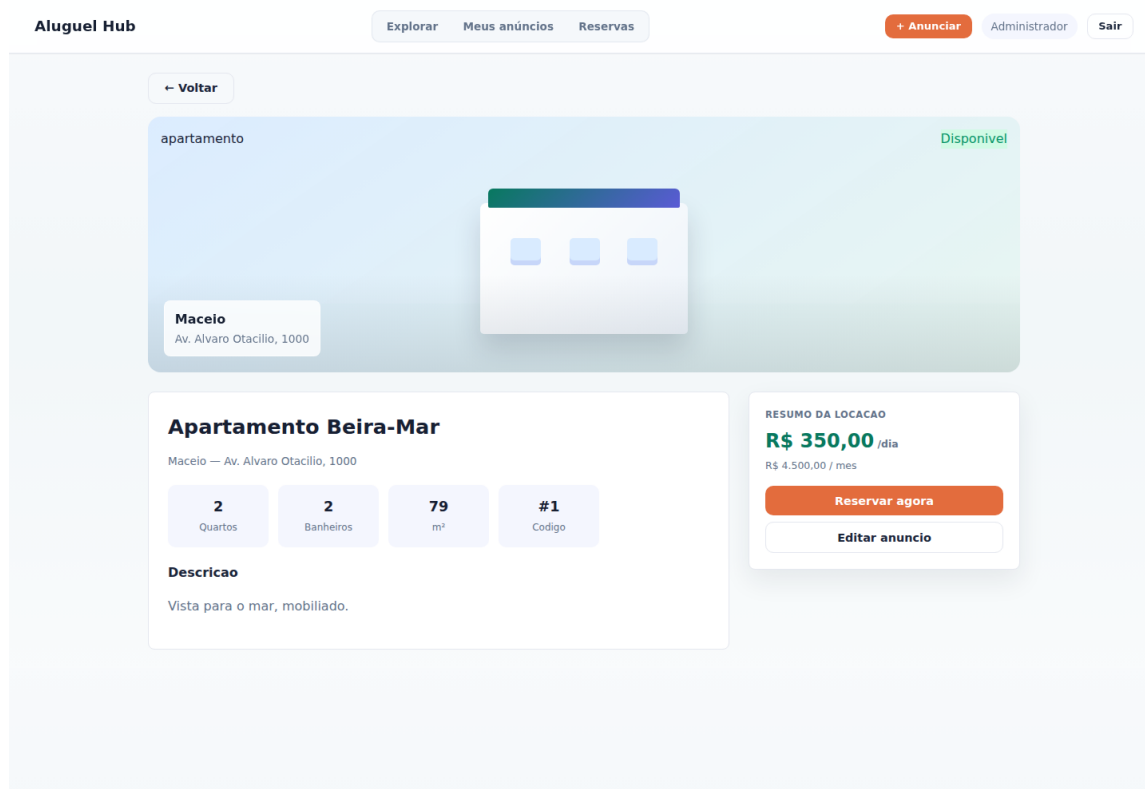


Figura 11: Tela de detalhes do imóvel e chamada para reserva.

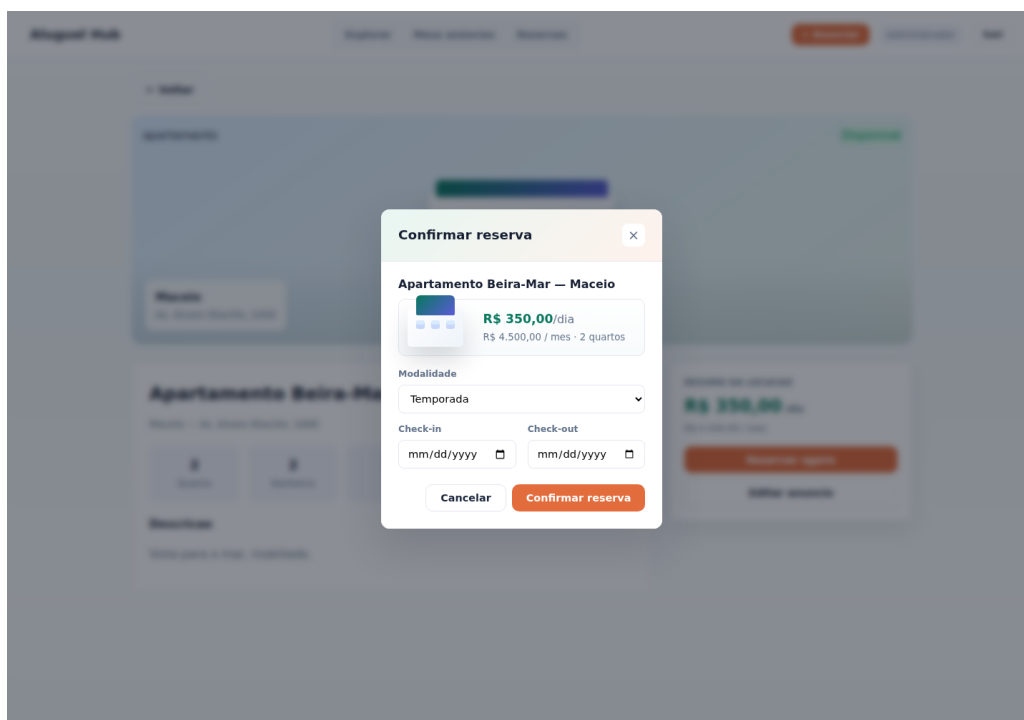


Figura 12: Modal de criação de reserva usando as modalidades do framework.

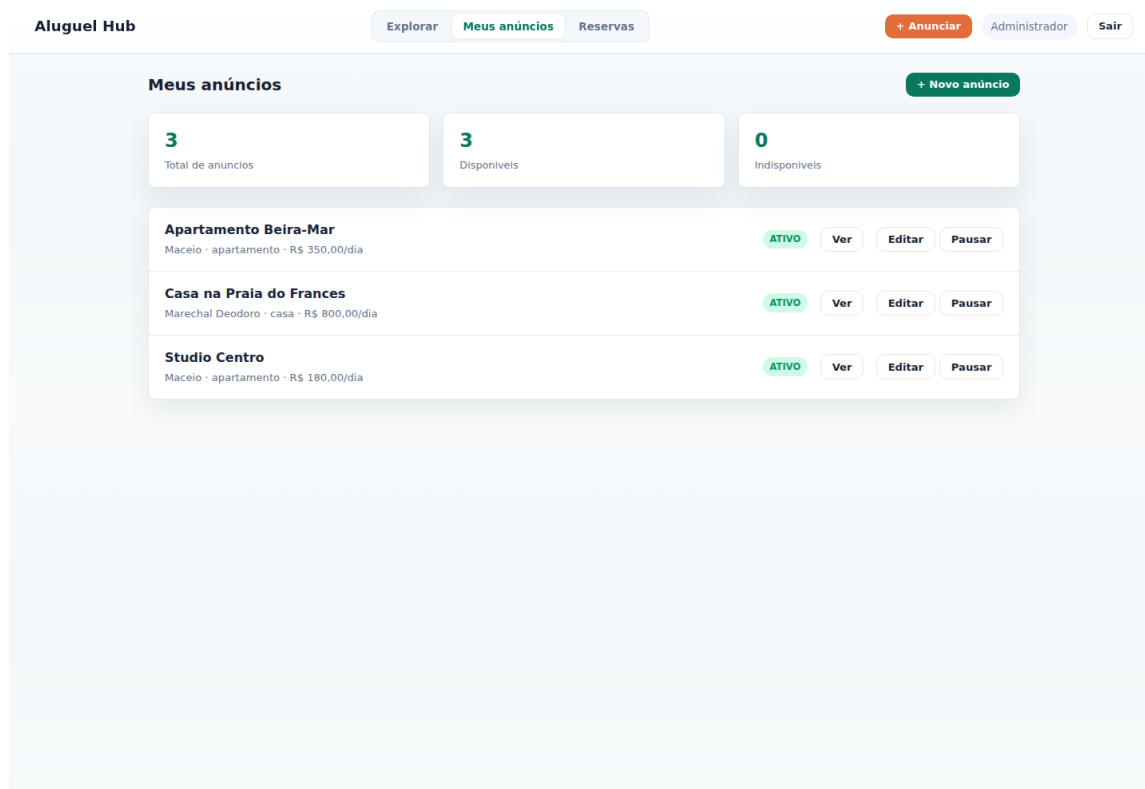
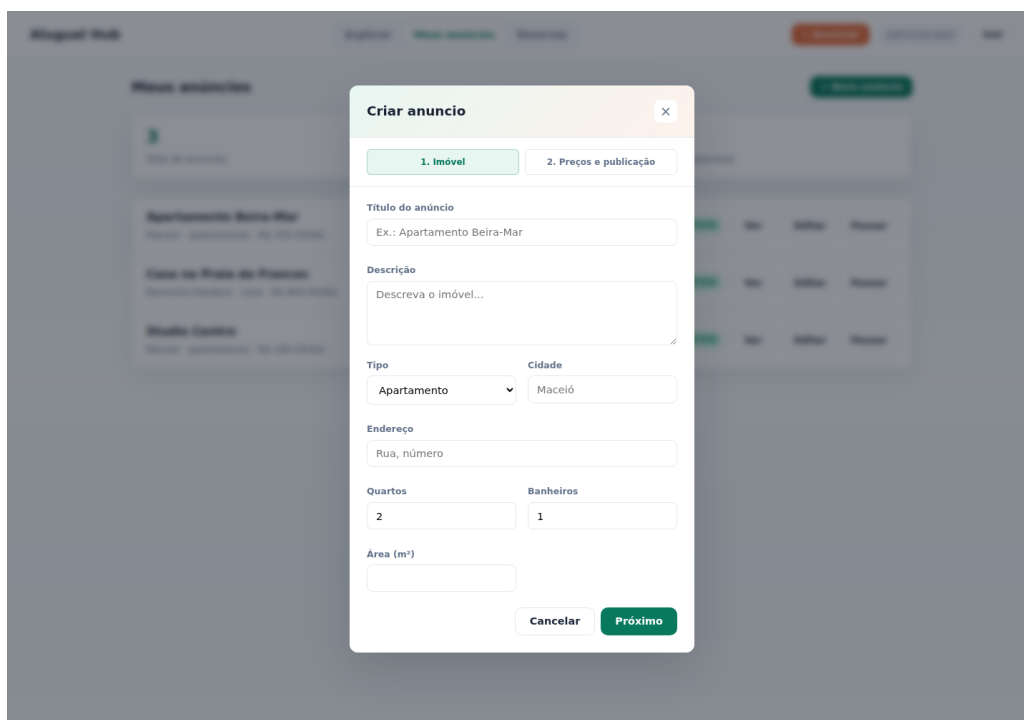
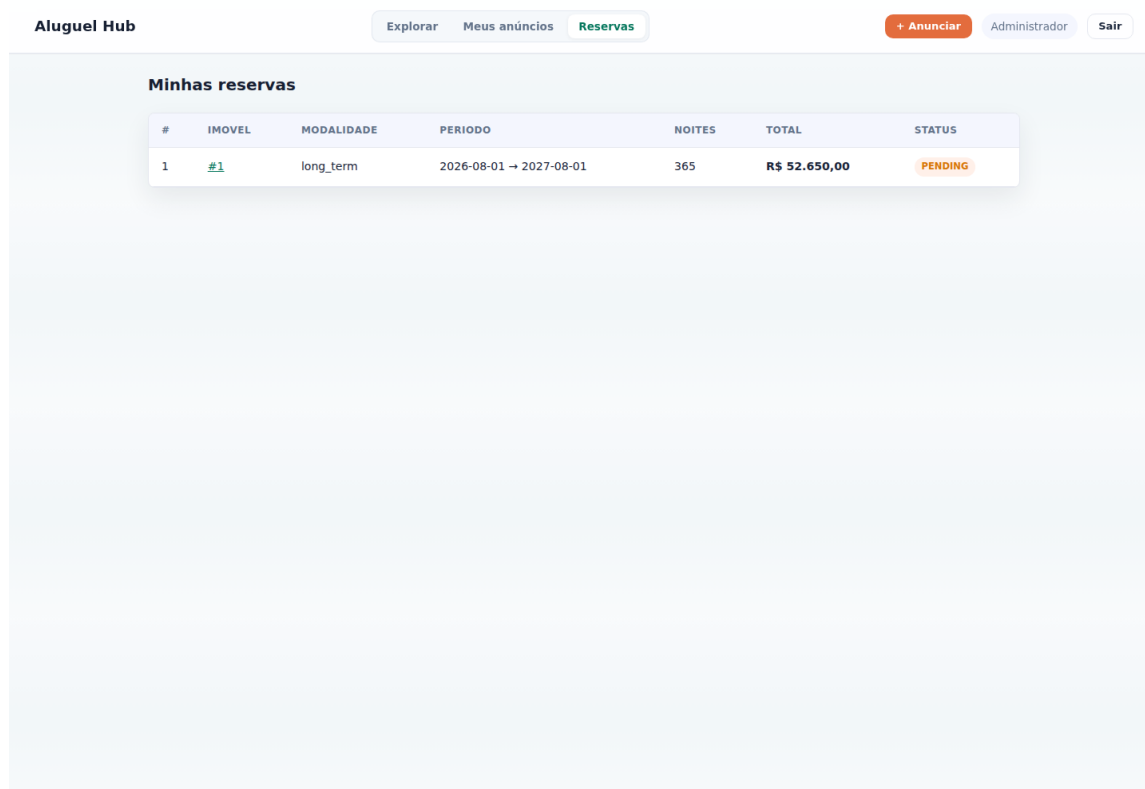


Figura 13: Tela de anúncios do usuário autenticado.



Modal de cadastro de novo anúncio. O formulário é dividido em duas etapas: 1. Imóvel e 2. Preços e publicação. A etapa 1 contém campos para: Título do anúncio (Ex.: Apartamento Beira-Mar), Descrição (Descreva o imóvel...), Tipo (dropdown com opção Apartamento), Cidade (Maceió), Endereço (Rua, número), Quartos (2), Banheiros (1) e Área (m²). Botões de Cancelar e Próximo estão na base.

Figura 14: Modal de cadastro de novo anúncio.



Tela de reservas com uma reserva de longa duração criada pelo framework. A interface mostra o menu "Reservas" selecionado. Abaixo, há uma tabela com uma única reserva pendente.

#	IMÓVEL	MODALIDADE	PERÍODO	NOITES	TOTAL	STATUS
1	#1	long_term	2026-08-01 → 2027-08-01	365	R\$ 52.650,00	PENDING

Figura 15: Tela de reservas com uma reserva de longa duração criada pelo framework.

12 Repositorio GitHub

O código completo do projeto, incluindo framework, aplicações concretas, microsserviços, scripts SQL, front-end e relatório, está disponível no GitHub:

<https://github.com/TheMarksan/aluguel-services-framework>

As instruções de execução estão no README.md: instalar dependências com Composer, criar os bancos com os scripts SQL, configurar os arquivos .env a partir dos .env.example e subir cada microsserviço em sua porta.

12.1 Observação sobre Vídeo

O requisito de vídeo de execução é aplicável apenas a frameworks para jogos. Como este projeto é um framework whitelabel para sistemas de aluguel de imóveis usando microsserviços, esse item não se aplica.

A Anexos: Código dos Microsserviços

Esta seção referencia o código documentado dos principais arquivos de cada microsserviço implementado. Os arquivos completos também estão no repositório GitHub.

A.1 API Gateway

```
<?php

declare(strict_types=1);

/**
 * Front Controller do API Gateway (porta 8003).
 *
 * Ponto de entrada único do ecossistema. Toda requisição do cliente passa por
 * aqui e é roteada (stateless) para o microsserviço adequado via cURL.
 */

use Gateway\Config;
use Gateway\HttpClient;
use Gateway Router;

$basePath = dirname(__DIR__);

// Autoloader: usa o do Composer se existir; senão, registra um PSR-4 mínimo.
$composerAutoload = $basePath . '/vendor/autoload.php';
if (is_file($composerAutoload)) {
    require $composerAutoload;
} else {
    spl_autoload_register(static function (string $class) use ($basePath): void {
        $prefix = 'Gateway\\';
        if (!str_starts_with($class, $prefix)) {
            return;
        }
        $relative = str_replace('\\', '/', substr($class, strlen($prefix)));
        $file = $basePath . '/src/' . $relative . '.php';
        if (is_file($file)) {
            require $file;
        }
    });
}
```

```

    });
}

Config::load($basePath);

$router = new Router(
    new HttpClient(Config::int('HTTP_TIMEOUT', 10)),
    $basePath . '/views'
);

$router->dispatch(
    $_SERVER['REQUEST_METHOD'] ?? 'GET',
    $_SERVER['REQUEST_URI'] ?? '/'
);

```

Listing 11: gateway/public/index.php

```

<?php

declare(strict_types=1);

namespace Gateway;

/**
 * Roteador central stateless do Gateway.
 *
 * Estrategia de roteamento:
 * - Rotas iniciadas em /api/{servico}/... sao encaminhadas (proxy) para o
 * microservico correspondente via cURL, preservando metodo, corpo e o
 * cabecalho Authorization (token JWT). Por ser stateless, nenhum estado
 * de sessao e mantido aqui: o token viaja em cada requisicao.
 * - Demais rotas (sem /api) sao tratadas como views (Blade/HTML).
 */
final class Router
{
    public function __construct(
        private readonly HttpClient $http,
        private readonly string $viewsPath
    ) {
    }

    public function dispatch(string $method, string $uri): void
    {
        $path = parse_url($uri, PHP_URL_PATH) ?? '/';
        $path = '/' . trim($path, '/');

        if (str_starts_with($path, '/api/')) {
            $this->handleApi($method, $path, $uri);
            return;
        }

        $this->handleView($path);
    }

    /**
     * Encaminha chamadas de API para o microservico correto.
     */
    private function handleApi(string $method, string $path, string $uri): void
    {
        $segments = explode('/', trim(substr($path, strlen('/api/')), '/'));
        $service = $segments[0] ?? '';
        $rest = implode('/', array_slice($segments, 1));

        $serviceMap = Config::serviceMap();
        if (!isset($serviceMap[$service])) {
            $this->json(404, [
                'error' => 'Not Found',
                'message' => "Servico desconhecido: '{$service}'.",
            ]);
            return;
        }

        $headers = $this->forwardableHeaders();
    }

```

```

$isProtected = ($service === 'imoveis' && in_array($method, ['POST', 'PUT', 'DELETE']))
|| ($service === 'reservas' && in_array($method, ['POST', 'PUT', 'DELETE']));

if ($isProtected) {
    $authUrl = $serviceMap['auth'] . '/validate';

    $authCheck = $this->http->forward('POST', $authUrl, '', $headers);

    if ($authCheck['status'] !== 200) {
        $this->json(401, ['error' => 'Não autorizado pelo Gateway. Token inválido ou ausente.']);
        return;
    }

    $claims = json_decode($authCheck['body'], true)['claims'] ?? [];

    if (isset($claims['sub'])) {
        $headers['X-User-Id'] = (string) $claims['sub'];
        $headers['X-User-Role'] = (string) ($claims['role'] ?? 'locatario');
    }
}

$query = parse_url($uri, PHP_URL_QUERY);
$targetUrl = rtrim($serviceMap[$service], '/') . '/' . $rest;
if ($query) {
    $targetUrl .= '?' . $query;
}

$body = file_get_contents('php://input') ?: '';
$response = $this->http->forward($method, $targetUrl, $body, $headers);

http_response_code($response['status']);
if (isset($response['content_type'])) {
    header('Content-Type: ' . $response['content_type']);
}
echo $response['body'];
}

/**
 * Renderiza views HTML do front-end (SPA via hash routing em home.php).
 * Rotas amigáveis /imovel, /meus-anuncios etc. também servem home.php.
 */
private function handleView(string $path): void
{
    $spaRoutes = ['', 'imovel', 'meus-anuncios', 'reservas'];
    $segment = $path === '/' ? '' : explode('/', trim($path, '/'))[0];

    if (in_array($segment, $spaRoutes, true)) {
        $file = $this->viewsPath . '/home.php';
        if (is_file($file)) {
            header('Content-Type: text/html; charset=utf-8');
            require $file;
            return;
        }
    }

    $view = $path === '/' ? 'home' : trim($path, '/');
    $file = $this->viewsPath . '/' . basename($view) . '.php';

    if (is_file($file)) {
        header('Content-Type: text/html; charset=utf-8');
        require $file;
        return;
    }

    http_response_code(404);
    header('Content-Type: text/html; charset=utf-8');
    echo '<h1>404 - Pagina nao encontrada</h1>';
}

/**
 * Cabecalhos repassados aos microservicos (mantendo o fluxo stateless).
 */

```

```

    * @return array<string,string>
    */
private function forwardableHeaders(): array
{
    $headers = [];
    $auth = $_SERVER['HTTP_AUTHORIZATION'] ?? ($_SERVER['REDIRECT_HTTP_AUTHORIZATION'] ?? '');
    if ($auth !== '') {
        $headers['Authorization'] = $auth;
    }
    return $headers;
}

/**
 * @param array<string,mixed> $data
 */
private function json(int $status, array $data): void
{
    http_response_code($status);
    header('Content-Type: application/json');
    echo json_encode($data, JSON_UNESCAPED_UNICODE);
}
}

```

Listing 12: gateway/src/Router.php

```

<?php

declare(strict_types=1);

namespace Gateway;

/**
 * Cliente HTTP minimalista baseado em cURL.
 *
 * Responsavel por toda a comunicacao interna do Gateway com os microservicos.
 * E intencionalmente "burro": apenas encaminha metodo, corpo e cabecalhos,
 * mantendo o Gateway stateless (nao guarda sessao nem estado entre chamadas).
 */
final class HttpClient
{
    public function __construct(private readonly int $timeout = 10)
    {
    }

    /**
     * Encaminha uma requisicao para um microservico interno.
     *
     * @param array<string,string> $headers Cabecalhos a repassar (ex.: Authorization)
     * @return array{status:int,body:string,content_type:string}
     */
    public function forward(string $method, string $url, string $body = '', array $headers = []): array
    {
        $ch = curl_init();

        $curlHeaders = ['Accept: application/json'];
        foreach ($headers as $name => $value) {
            $curlHeaders[] = "{$name}: {$value}";
        }
        if ($body !== '') {
            $curlHeaders[] = 'Content-Type: application/json';
        }

        curl_setopt_array($ch, [
            CURLOPT_URL => $url,
            CURLOPT_CUSTOMREQUEST => strtoupper($method),
            CURLOPT_RETURNTRANSFER => true,
            CURLOPT_HEADER => false,
            CURLOPT_CONNECTTIMEOUT => $this->timeout,
            CURLOPT_TIMEOUT => $this->timeout,
            CURLOPT_HTTPHEADER => $curlHeaders,
        ]);

        if ($body !== '') {

```

```

        curl_setopt($ch, CURLOPT_POSTFIELDS, $body);
    }

    $response = curl_exec($ch);
    $errorNo = curl_errno($ch);
    $errorMsg = curl_error($ch);
    $statusCode = (int) curl_getinfo($ch, CURLINFO_HTTP_CODE);
    $contentType = (string) curl_getinfo($ch, CURLINFO_CONTENT_TYPE);
    curl_close($ch);

    if ($errorNo !== 0) {
        return [
            'status' => 502,
            'body' => json_encode([
                'error' => 'Bad Gateway',
                'message' => "Falha ao contatar o microservico: {$errorMsg}",
            ], JSON_UNESCAPED_UNICODE),
            'content_type' => 'application/json',
        ];
    }

    return [
        'status' => $statusCode,
        'body' => is_string($response) ? $response : '',
        'content_type' => $contentType !== '' ? $contentType : 'application/json',
    ];
}
}

```

Listing 13: gateway/src/HttpClient.php

```

<?php

declare(strict_types=1);

namespace Gateway;

/**
 * Carregador simples de configuracao a partir do arquivo .env.
 *
 * Mantem o Gateway desacoplado das URLs internas dos microservicos,
 * permitindo reconfigurar o ecossistema sem alterar codigo.
 */
final class Config
{
    /** @var array<string,string> */
    private static array $values = [];

    private static bool $loaded = false;

    public static function load(string $basePath): void
    {
        if (self::$loaded) {
            return;
        }

        $envFile = $basePath . '/.env';
        if (is_file($envFile)) {
            foreach (file($envFile, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES) as $line) {
                $line = trim($line);
                if ($line === '' || str_starts_with($line, '#')) {
                    continue;
                }
                [$key, $value] = array_pad(explode('=', $line, 2), 2, '');
                self::$values[trim($key)] = trim($value);
            }

            self::$loaded = true;
        }

        public static function get(string $key, string $default = ''): string
        {

```

```

        return self::$values[$key] ?? getenv($key) ?: $default;
    }

    public static function int(string $key, int $default = 0): int
    {
        $value = self::get($key, (string) $default);
        return $value === '' ? $default : (int) $value;
    }

    /**
     * Mapa logico: prefixo de rota -> URL base do microservico.
     *
     * @return array<string,string>
     */
    public static function serviceMap(): array
    {
        return [
            'auth' => self::get('MS_AUTH_URL', 'http://localhost:8000'),
            'imoveis' => self::get('MS_CATALOGO_URL', 'http://localhost:8001'),
            'reservas' => self::get('MS_RESERVAS_URL', 'http://localhost:8002'),
        ];
    }
}

```

Listing 14: gateway/src/Config.php

A.2 Microservico ms-auth

```

<?php

declare(strict_types=1);

/**
 * Front Controller do ms-auth (porta 8000).
 *
 * Servico de autenticao JWT. E acessado internamente pelo Gateway,
 * mas tambem pode ser testado diretamente em http://localhost:8000.
 */

use MsAuth\AuthController;
use MsAuth\Config;
use MsAuth\Database;
use MsAuth\UserRepository;

$basePath = dirname(__DIR__);

$composerAutoload = $basePath . '/vendor/autoload.php';
if (is_file($composerAutoload)) {
    require $composerAutoload;
} else {
    spl_autoload_register(static function (string $class) use ($basePath): void {
        $prefix = 'MsAuth\\';
        if (!str_starts_with($class, $prefix)) {
            return;
        }
        $relative = str_replace('\\', '/', substr($class, strlen($prefix)));
        $file = $basePath . '/src/' . $relative . '.php';
        if (is_file($file)) {
            require $file;
        }
    });
}

Config::load($basePath);

$method = $_SERVER['REQUEST_METHOD'] ?? 'GET';
$path = '/' . trim(parse_url($_SERVER['REQUEST_URI'] ?? '', PHP_URL_PATH) ?: '/', '/');
$auth = $_SERVER['HTTP_AUTHORIZATION'] ?? ($_SERVER['REDIRECT_HTTP_AUTHORIZATION'] ?? '');

/** @return array<string,mixed> */

```



```

$readJsonBody = static function (): array {
    $raw = file_get_contents('php://input') ?? '';
    $data = json_decode($raw, true);
    return is_array($data) ? $data : [];
};

header('Content-Type: application/json; charset=utf-8');

try {
    $controller = new AuthController(new UserRepository(Database::connection()));

    match (true) {
        $method === 'POST' && $path === '/register' => $controller->register($readJsonBody()),
        $method === 'POST' && $path === '/login' => $controller->login($readJsonBody()),
        $method === 'POST' && $path === '/logout' => $controller->logout($auth),
        $method === 'POST' && $path === '/refresh' => $controller->refresh($auth),
        $method === 'GET' && $path === '/me' => $controller->me($auth),
        $method === 'POST' && $path === '/validate' => $controller->validate($auth),
        $method === 'GET' && $path === '/health' => print(json_encode(['service' => 'ms-auth', 'status' =>
            'ok'])),
        default => (static function (): void {
            http_response_code(404);
            echo json_encode(['error' => 'Rota nao encontrada no ms-auth.'], JSON_UNESCAPED_UNICODE);
        })(),
    };
} catch (\Throwable $e) {
    http_response_code(500);
    echo json_encode(['error' => 'Erro interno no ms-auth.', 'detail' => $e->getMessage()],
        JSON_UNESCAPED_UNICODE);
}

```

Listing 15: ms-auth/public/index.php

```

<?php

declare(strict_types=1);

namespace MsAuth;

/**
 * Controlador de autenticacao do ms-auth.
 *
 * Endpoints expostos (consumidos via Gateway em /api/auth/*):
 * POST /register -> cria usuario
 * POST /login -> autentica e devolve token JWT
 * GET /me -> retorna dados do usuario do token (Authorization)
 * POST /validate -> valida um token (uso interno pelos demais servicos)
 */
final class AuthController
{
    public function __construct(private readonly UserRepository $users)
    {
    }

    /**
     * @param array<string,mixed> $input
     */
    public function register(array $input): void
    {
        $name = $input['name'] ?? '';
        $email = $input['email'] ?? '';
        $password = $input['password'] ?? '';
        $role = $input['role'] ?? 'locatario';

        if (!$name || !$email || !$password) {
            $this->json(400, ['error' => 'Dados incompletos. Nome, email e password sao obrigatorios.']);
            return;
        }

        if ($role === 'admin') {
            $this->json(403, ['error' => 'Nao e permitido registrar contas com o papel de administrador.']);
            return;
        }
    }
}

```

```

        if (!in_array($role, ['locador', 'locatario'])) {
            $this->json(400, ['error' => 'Papel (role) invalido. Escolha locador ou locatario.']);
            return;
        }

        $hash = password_hash($password, PASSWORD_BCRYPT);

        try {
            $id = $this->users->create($name, $email, $hash, $role);
            $this->json(201, [
                'message' => 'Utilizador registado com sucesso.',
                'data' => ['id' => $id, 'name' => $name, 'email' => $email, 'role' => $role]
            ]);
        } catch (\PDOException $e) {
            if ($e->getCode() == 23000) {
                $this->json(409, ['error' => 'Este email ja se encontra registado no sistema.']);
            } else {
                $this->json(500, ['error' => 'Erro interno na base de dados.']);
            }
        }
    }

    /**
     * @param array<string,mixed> $input
     */
    public function login(array $input): void
    {
        $email = trim((string) ($input['email'] ?? ''));
        $password = (string) ($input['password'] ?? '');

        $user = $email !== '' ? $this->users->findByEmail($email) : null;

        if ($user === null || !password_verify($password, (string) $user['password_hash'])) {
            $this->json(401, ['error' => 'Credenciais invalidas.']);
            return;
        }

        $token = Jwt::encode(
            [
                'sub' => (int) $user['id'],
                'name' => $user['name'],
                'email' => $user['email'],
                'role' => $user['role'],
            ],
            Config::get('JWT_SECRET'),
            Config::get('JWT_ISSUER', 'ms-auth'),
            Config::int('JWT_TTL', 3600)
        );

        $this->json(200, [
            'token' => $token,
            'token_type' => 'Bearer',
            'expires_in' => Config::int('JWT_TTL', 3600),
            'user' => [
                'id' => (int) $user['id'],
                'name' => $user['name'],
                'email' => $user['email'],
                'role' => $user['role'],
            ],
        ]);
    }

    public function logout(string $authHeader): void
    {
        $token = str_replace('Bearer ', '', $authHeader);
        if (!$token) {
            $this->json(400, ['error' => 'Token ausente.']);
            return;
        }

        $this->users->revokeToken($token);
        $this->json(200, ['message' => 'Logout efetuado com sucesso. Token revogado.']);
    }

```

```

}

public function refresh(string $authHeader): void
{
    $claims = $this->resolveClaims($authHeader);
    if (!$claims) {
        return;
    }

    $oldToken = str_replace('Bearer ', '', $authHeader);
    $this->users->revokeToken($oldToken);

    $secret = getenv('JWT_SECRET');
    $issuer = getenv('JWT_ISSUER');
    $ttl = (int) getenv('JWT_TTL') ?: 3600;

    $newClaims = [
        'sub' => $claims['sub'],
        'role' => $claims['role']
    ];

    $newToken = Jwt::encode($newClaims, $secret, $issuer, $ttl);
    $this->json(200, ['token' => $newToken]);
}

public function me(string $authorizationHeader): void
{
    $claims = $this->resolveClaims($authorizationHeader);
    if ($claims === null) {
        $this->json(401, ['error' => 'Token ausente ou invalido.']);
        return;
    }

    $this->json(200, [
        'id' => $claims['sub'] ?? null,
        'name' => $claims['name'] ?? null,
        'email' => $claims['email'] ?? null,
        'role' => $claims['role'] ?? null,
    ]);
}

/**
 * Validacao de token para uso interno por outros microservicos.
 */
public function validate(string $authorizationHeader): void
{
    $token = str_replace('Bearer ', '', $authorizationHeader);

    if ($token && $this->users->isTokenRevoked($token)) {
        $this->json(401, ['error' => 'Token revogado. Faca login novamente.']);
        return;
    }

    $claims = $this->resolveClaims($authorizationHeader);
    if ($claims === null) {
        $this->json(401, ['valid' => false]);
        return;
    }

    $this->json(200, ['valid' => true, 'claims' => $claims]);
}

/**
 * @return array<string,mixed>/null
 */
private function resolveClaims(string $authorizationHeader): ?array
{
    if (!preg_match('/Bearer\s+(\S+)/i', $authorizationHeader, $m)) {
        return null;
    }

    return Jwt::decode($m[1], Config::get('JWT_SECRET'));
}

```

```

/**
 * @param array<string,mixed> $data
 */
private function json(int $status, array $data): void
{
    http_response_code($status);
    header('Content-Type: application/json; charset=utf-8');
    echo json_encode($data, JSON_UNESCAPED_UNICODE);
}
}

```

Listing 16: ms-auth/src/AuthController.php

```

<?php

declare(strict_types=1);

namespace MsAuth;

use PDO;

/**
 * Acesso a dados da tabela users (db_auth).
 */
final class UserRepository
{
    public function __construct(private readonly PDO $pdo)
    {
    }

    /**
     * @return array<string,mixed>/null
     */
    public function findByEmail(string $email): ?array
    {
        $stmt = $this->pdo->prepare('SELECT * FROM users WHERE email = :email LIMIT 1');
        $stmt->execute(['email' => $email]);
        $user = $stmt->fetch();

        return $user ?: null;
    }

    public function emailExists(string $email): bool
    {
        $stmt = $this->pdo->prepare('SELECT 1 FROM users WHERE email = :email LIMIT 1');
        $stmt->execute(['email' => $email]);

        return (bool) $stmt->fetchColumn();
    }

    /**
     * @return int ID do usuario criado.
     */
    public function create(string $name, string $email, string $passwordHash, string $role): int
    {
        $stmt = $this->pdo->prepare(
            'INSERT INTO users (name, email, password_hash, role)
             VALUES (:name, :email, :password_hash, :role)'
        );
        $stmt->execute([
            'name' => $name,
            'email' => $email,
            'password_hash' => $passwordHash,
            'role' => $role,
        ]);

        return (int) $this->pdo->lastInsertId();
    }

    public function revokeToken(string $token): void
    {
        $stmt = $this->pdo->prepare("INSERT IGNORE INTO revoked_tokens (token) VALUES (:token)");
    }
}

```

```

        $stmt->execute(['token' => $token]);
    }

    public function isTokenRevoked(string $token): bool
    {
        $stmt = $this->pdo->prepare("SELECT 1 FROM revoked_tokens WHERE token = :token");
        $stmt->execute(['token' => $token]);
        return (bool) $stmt->fetchColumn();
    }
}

```

Listing 17: ms-auth/src/UserRepository.php

```

<?php

declare(strict_types=1);

namespace MsAuth;

/**
 * Implementacao minima de JWT (JSON Web Token) com algoritmo HS256.
 *
 * Evita dependencias externas para fins didaticos, mantendo o microservico
 * autocontido. Em producao, considere a biblioteca firebase/php-jwt.
 */
final class Jwt
{
    /**
     * Gera um token assinado.
     *
     * @param array<string,mixed> $claims Dados adicionais (ex.: sub, role)
     */
    public static function encode(array $claims, string $secret, string $issuer, int $ttl): string
    {
        $now = time();
        $header = ['alg' => 'HS256', 'typ' => 'JWT'];
        $payload = array_merge($claims, [
            'iss' => $issuer,
            'iat' => $now,
            'exp' => $now + $ttl,
        ]);

        $segments = [
            self::base64UrlEncode((string) json_encode($header)),
            self::base64UrlEncode((string) json_encode($payload)),
        ];

        $signature = self::sign(implode('.', $segments), $secret);
        $segments[] = self::base64UrlEncode($signature);

        return implode('.', $segments);
    }

    /**
     * Decodifica e valida assinatura + expiracao.
     *
     * @return array<string,mixed>/null Claims se valido; null caso contrario.
     */
    public static function decode(string $token, string $secret): ?array
    {
        $parts = explode('.', $token);
        if (count($parts) !== 3) {
            return null;
        }

        [$encodedHeader, $encodedPayload, $encodedSignature] = $parts;

        $expected = self::sign("{\"$encodedHeader\".\"$encodedPayload\"", $secret);
        $provided = self::base64UrlDecode($encodedSignature);

        if (!hash_equals($expected, $provided)) {
            return null;
        }
    }
}

```

```

        $payload = json_decode(self::base64UrlDecode($encodedPayload), true);
        if (!is_array($payload)) {
            return null;
        }

        if (isset($payload['exp']) && time() >= (int) $payload['exp']) {
            return null;
        }

        return $payload;
    }

    private static function sign(string $data, string $secret): string
    {
        return hash_hmac('sha256', $data, $secret, true);
    }

    private static function base64UrlEncode(string $data): string
    {
        return rtrim(strtr(base64_encode($data), '+/', '-_'), '=');
    }

    private static function base64UrlDecode(string $data): string
    {
        $remainder = strlen($data) % 4;
        if ($remainder > 0) {
            $data .= str_repeat('=', 4 - $remainder);
        }
        return (string) base64_decode(strtr($data, '-_', '+/'));
    }
}

```

Listing 18: ms-auth/src/Jwt.php

```

<?php

declare(strict_types=1);

namespace MsAuth;

use PDO;
use PDOException;
use RuntimeException;

/**
 * Fabrica de conexao PDO com o banco db_auth.
 */
final class Database
{
    private static ?PDO $connection = null;

    public static function connection(): PDO
    {
        if (self::$connection instanceof PDO) {
            return self::$connection;
        }

        $dsn = sprintf(
            'mysql:host=%s;port=%s;dbname=%s;charset=utf8mb4',
            Config::get('DB_HOST', '127.0.0.1'),
            Config::get('DB_PORT', '3306'),
            Config::get('DB_NAME', 'db_auth')
        );

        try {
            self::$connection = new PDO(
                $dsn,
                Config::get('DB_USER', 'root'),
                Config::get('DB_PASS', ''),
                [
                    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                ]
            );
        } catch (PDOException $e) {
            throw new RuntimeException($e->getMessage());
        }
    }
}

```

```

        PDO::ATTR_EMULATE_PREPARES => false,
    ]
);
} catch (PDOException $e) {
    throw new RuntimeException('Falha ao conectar ao banco db_auth: ' . $e->getMessage());
}

return self::$connection;
}
}

```

Listing 19: ms-auth/src/Database.php

```

-- =====
-- Banco: db_auth (Microservico ms-auth - porta 8000)
-- Responsavel pela autenticao e emissao/validacao de tokens JWT.
-- Princípio: Database per Service (isolamento de dados).
-- =====

CREATE DATABASE IF NOT EXISTS db_auth
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE db_auth;

-- -----
-- Tabela de usuarios
-- -----

CREATE TABLE IF NOT EXISTS users (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name VARCHAR(120) NOT NULL,
    email VARCHAR(180) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role ENUM('admin','locador','locatario') NOT NULL DEFAULT 'locatario',
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_users_email (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS revoked_tokens (
    token VARCHAR(512) PRIMARY KEY,
    revoked_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Listing 20: ms-auth/sql/db_auth.sql

A.3 Microservico ms-catalogo

```

<?php

declare(strict_types=1);

/**
 * Front Controller do ms-catalogo (porta 8001).
 *
 * O Gateway encaminha /api/imoveis/* para ca, removendo o prefixo. Assim:
 * /api/imoveis -> "/" (colecção)
 * /api/imoveis/5 -> "/5" (item)
 */

use MsCatalogo\CatalogController;
use MsCatalogo\Config;
use MsCatalogo\Database;
use MsCatalogo\PropertyRepository;

$basePath = dirname(__DIR__);

$composerAutoload = $basePath . '/vendor/autoload.php';

```

```

if (is_file($composerAutoload)) {
    require $composerAutoload;
} else {
    spl_autoload_register(static function (string $class) use ($basePath): void {
        $prefix = 'MsCatalogo\\';
        if (!str_starts_with($class, $prefix)) {
            return;
        }
        $relative = str_replace('\\', '/', substr($class, strlen($prefix)));
        $file = $basePath . '/src/' . $relative . '.php';
        if (is_file($file)) {
            require $file;
        }
    });
}

Config::load($basePath);

$method = $_SERVER['REQUEST_METHOD'] ?? 'GET';
$path = '/' . trim(parse_url($_SERVER['REQUEST_URI'] ?? '/', PHP_URL_PATH) ?: '/', '/');

/** @return array<string,mixed> */
$readJsonBody = static function (): array {
    $raw = file_get_contents('php://input') ?: '';
    $data = json_decode($raw, true);
    return is_array($data) ? $data : [];
};

header('Content-Type: application/json; charset=utf-8');

// Extraí um eventual id numerico do path (ex.: /5).
$id = null;
if (preg_match('#~/(\d+)#', $path, $m)) {
    $id = (int) $m[1];
}

try {
    $controller = new CatalogController(new PropertyRepository(Database::connection()));

    match (true) {
        $method === 'GET' && $path === '/health' => print(json_encode(['service' => 'ms-catalogo', 'status'
        => 'ok'])),
        $method === 'GET' && $path === '/' => $controller->index($_GET),
        $method === 'POST' && $path === '/' => $controller->store($readJsonBody()),
        $method === 'GET' && $id !== null => $controller->show($id),
        $method === 'PUT' && $id !== null => $controller->update($id, $readJsonBody()),
        $method === 'DELETE' && $id !== null => $controller->destroy($id),
        default => (static function (): void {
            http_response_code(404);
            echo json_encode(['error' => 'Rota nao encontrada no ms-catalogo.'], JSON_UNESCAPED_UNICODE);
        })(),
    };
} catch (\Throwable $e) {
    http_response_code(500);
    echo json_encode(['error' => 'Erro interno no ms-catalogo.', 'detail' => $e->getMessage()],
        JSON_UNESCAPED_UNICODE);
}

```

Listing 21: ms-catalogo/public/index.php

```

<?php

declare(strict_types=1);

namespace MsCatalogo;

final class CatalogController
{
    public function __construct(private readonly PropertyRepository $properties)
    {
    }

    private function getAuthUser(): ?array

```



```

{
    $id = $_SERVER['HTTP_X_USER_ID'] ?? null;
    $role = $_SERVER['HTTP_X_USER_ROLE'] ?? null;
    return $id ? ['id' => (int) $id, 'role' => $role] : null;
}

public function index(array $filters): void
{
    $this->json(200, ['data' => $this->properties->all($filters)]);
}

public function show(int $id): void
{
    $property = $this->properties->find($id);
    if ($property === null) {
        $this->json(404, ['error' => 'Imovel nao encontrado.']);
        return;
    }
    $this->json(200, ['data' => $property]);
}

public function store(array $input): void
{
    $user = $this->getAuthUser();
    if (!$user) {
        $this->json(401, ['error' => 'Acesso negado. Requisicao nao validada pelo Gateway.']);
        return;
    }

    if (!in_array($user['role'], ['admin', 'locador'], true)) {
        $this->json(403, ['error' => 'Apenas locadores ou administradores podem anunciar imoveis.']);
        return;
    }

    $error = $this->validate($input);
    if ($error !== null) {
        $this->json(422, ['error' => $error]);
        return;
    }

    $input['owner_id'] = $user['id'];

    $id = $this->properties->create($input);
    $this->json(201, ['data' => $this->properties->find($id)]);
}

public function update(int $id, array $input): void
{
    $user = $this->getAuthUser();
    if (!$user) {
        $this->json(401, ['error' => 'Acesso negado. Requisicao nao validada pelo Gateway.']);
        return;
    }

    $property = $this->properties->find($id);
    if ($property === null) {
        $this->json(404, ['error' => 'Imovel nao encontrado.']);
        return;
    }

    if ($user['role'] !== 'admin' && (int)$property['owner_id'] !== $user['id']) {
        $this->json(403, ['error' => 'Acesso negado. Apenas o dono do anuncio pode edita-lo.']);
        return;
    }

    $error = $this->validate($input);
    if ($error !== null) {
        $this->json(422, ['error' => $error]);
        return;
    }

    $input['owner_id'] = $property['owner_id'];

```

```

        $this->properties->update($id, $input);
        $this->json(200, ['data' => $this->properties->find($id)]);
    }

    public function destroy(int $id): void
    {
        $user = $this->getAuthUser();
        if (!$user) {
            $this->json(401, ['error' => 'Acesso negado.']);
            return;
        }

        $property = $this->properties->find($id);
        if ($property === null) {
            $this->json(404, ['error' => 'Imovel nao encontrado.']);
            return;
        }

        if ($user['role'] !== 'admin' && (int)$property['owner_id'] !== $user['id']) {
            $this->json(403, ['error' => 'Acesso negado. Apenas o dono do anuncio pode apaga-lo.']);
            return;
        }

        $this->properties->delete($id);
        $this->json(200, ['message' => 'Imovel removido com sucesso.']);
    }

    private function validate(array $input): ?string
    {
        if (trim((string) ($input['title'] ?? '')) === '') {
            return 'O campo title e obrigatorio.';
        }
        if (trim((string) ($input['city'] ?? '')) === '') {
            return 'O campo city e obrigatorio.';
        }
        if (trim((string) ($input['address'] ?? '')) === '') {
            return 'O campo address e obrigatorio.';
        }
        if (isset($input['type']) && !in_array($input['type'], ['casa', 'apartamento'], true)) {
            return 'O tipo deve ser "casa" ou "apartamento".';
        }
        if (isset($input['daily_price']) && (float)$input['daily_price'] < 0) {
            return 'O preco da diaria nao pode ser negativo.';
        }
        if (isset($input['monthly_price']) && (float)$input['monthly_price'] < 0) {
            return 'O preco mensal nao pode ser negativo.';
        }
        return null;
    }

    private function json(int $status, array $data): void
    {
        http_response_code($status);
        header('Content-Type: application/json; charset=utf-8');
        echo json_encode($data, JSON_UNESCAPED_UNICODE);
    }
}

```

Listing 22: ms-catalogo/src/CatalogController.php

```

<?php

declare(strict_types=1);

namespace MsCatalogo;

use PDO;

/**
 * Acesso a dados da tabela properties (db_catalogo).
 */
final class PropertyRepository
{

```

```

public function __construct(private readonly PDO $pdo)
{
}

/**
 * Lista imoveis com filtros opcionais, paginacao e ordenacao.
 *
 * @param array<string,string> $filters
 * @return array<int,array<string,mixed>>
 */
public function all(array $filters = []): array
{
    $sql = 'SELECT * FROM properties';
    $conditions = [];
    $params = [];

    if (!empty($filters['city'])) {
        $conditions[] = 'city = :city';
        $params['city'] = $filters['city'];
    }
    if (!empty($filters['type'])) {
        $conditions[] = 'type = :type';
        $params['type'] = $filters['type'];
    }
    if (isset($filters['available']) && $filters['available'] !== '') {
        $conditions[] = 'available = :available';
        $params['available'] = (int) ((bool) $filters['available']);
    }

    if ($conditions !== []) {
        $sql .= ' WHERE ' . implode(' AND ', $conditions);
    }

    $allowedSorts = ['created_at', 'daily_price', 'monthly_price'];
    $sortBy = in_array($filters['sort_by'] ?? '', $allowedSorts, true) ? $filters['sort_by'] : '
    ↪ created_at';
    $sortDir = strtoupper($filters['sort_dir'] ?? '') === 'ASC' ? 'ASC' : 'DESC';
    $sql .= " ORDER BY {$sortBy} {$sortDir}";

    $page = max(1, (int) ($filters['page'] ?? 1));
    $perPage = max(1, min(100, (int) ($filters['per_page'] ?? 10)));
    $offset = ($page - 1) * $perPage;

    $sql .= " LIMIT {$perPage} OFFSET {$offset}";

    $stmt = $this->pdo->prepare($sql);
    $stmt->execute($params);

    return $stmt->fetchAll();
}

/**
 * @return array<string,mixed>/null
 */
public function find(int $id): ?array
{
    $stmt = $this->pdo->prepare('SELECT * FROM properties WHERE id = :id LIMIT 1');
    $stmt->execute(['id' => $id]);
    $row = $stmt->fetch();

    return $row ? : null;
}

/**
 * @param array<string,mixed> $data
 */
public function create(array $data): int
{
    $stmt = $this->pdo->prepare(
        'INSERT INTO properties
        (owner_id, title, description, type, city, address, bedrooms, bathrooms, area_m2,
        ↪ daily_price, monthly_price, available)
        VALUES

```

```

        (:owner_id, :title, :description, :type, :city, :address, :bedrooms, :bathrooms, :area_m2, :
        ↪ daily_price, :monthly_price, :available)'
    );
    $stmt->execute($this->bind($data));

    return (int) $this->pdo->lastInsertId();
}

/**
 * @param array<string,mixed> $data
 */
public function update(int $id, array $data): bool
{
    $params = $this->bind($data);
    $params['id'] = $id;

    $stmt = $this->pdo->prepare(
        'UPDATE properties SET
         owner_id = :owner_id, title = :title, description = :description, type = :type,
         city = :city, address = :address, bedrooms = :bedrooms, bathrooms = :bathrooms,
         area_m2 = :area_m2, daily_price = :daily_price, monthly_price = :monthly_price, available =
        ↪ :available
        WHERE id = :id'
    );
    $stmt->execute($params);

    return $stmt->rowCount() >= 0;
}

public function delete(int $id): bool
{
    $stmt = $this->pdo->prepare('DELETE FROM properties WHERE id = :id');
    $stmt->execute(['id' => $id]);

    return $stmt->rowCount() > 0;
}

/**
 * Normaliza os campos para bind seguro.
 *
 * @param array<string,mixed> $data
 * @return array<string,mixed>
 */
private function bind(array $data): array
{
    return [
        'owner_id' => isset($data['owner_id']) ? (int) $data['owner_id'] : null,
        'title' => (string) ($data['title'] ?? ''),
        'description' => (string) ($data['description'] ?? ''),
        'type' => in_array($data['type'] ?? '', ['casa', 'apartamento'], true) ? $data['type'] : '
        ↪ apartamento',
        'city' => (string) ($data['city'] ?? ''),
        'address' => (string) ($data['address'] ?? ''),
        'bedrooms' => (int) ($data['bedrooms'] ?? 1),
        'bathrooms' => (int) ($data['bathrooms'] ?? 1),
        'area_m2' => (float) ($data['area_m2'] ?? 0),
        'daily_price' => (float) ($data['daily_price'] ?? 0),
        'monthly_price' => (float) ($data['monthly_price'] ?? 0),
        'available' => (int) ((bool) ($data['available'] ?? true)),
    ];
}
}

```

Listing 23: ms-catalogo/src/PropertyRepository.php

```

<?php

declare(strict_types=1);

namespace MsCatalogo;

use PDO;
use PDOException;

```

```

use RuntimeException;

/**
 * Fabrica de conexao PDO com o banco db_catalogo.
 */
final class Database
{
    private static ?PDO $connection = null;

    public static function connection(): PDO
    {
        if (self::$connection instanceof PDO) {
            return self::$connection;
        }

        $dsn = sprintf(
            'mysql:host=%s;port=%s;dbname=%s;charset=utf8mb4',
            Config::get('DB_HOST', '127.0.0.1'),
            Config::get('DB_PORT', '3306'),
            Config::get('DB_NAME', 'db_catalogo')
        );

        try {
            self::$connection = new PDO(
                $dsn,
                Config::get('DB_USER', 'root'),
                Config::get('DB_PASS', ''),
                [
                    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                    PDO::ATTR_EMULATE_PREPARES => false,
                ]
            );
        } catch (PDOException $e) {
            throw new RuntimeException('Falha ao conectar ao banco db_catalogo: ' . $e->getMessage());
        }

        return self::$connection;
    }
}

```

Listing 24: ms-catalogo/src/Database.php

```

-- =====
-- Banco: db_catalogo (Microservico ms-catalogo - porta 8001)
-- Cadastro e consulta de imoveis (casas e apartamentos).
-- Princípio: Database per Service (isolamento de dados).
-- =====

CREATE DATABASE IF NOT EXISTS db_catalogo
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE db_catalogo;

-- -----
-- Tabela de imoveis
-- owner_id referencia logicamente users.id do db_auth (sem FK fisica,
-- pois cada microservico possui banco isolado).
-- -----

CREATE TABLE IF NOT EXISTS properties (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    owner_id BIGINT UNSIGNED NULL,
    title VARCHAR(160) NOT NULL,
    description TEXT NULL,
    type ENUM('casa','apartamento') NOT NULL DEFAULT 'apartamento',
    city VARCHAR(120) NOT NULL,
    address VARCHAR(255) NOT NULL,
    bedrooms TINYINT UNSIGNED NOT NULL DEFAULT 1,
    bathrooms TINYINT UNSIGNED NOT NULL DEFAULT 1,
    area_m2 DECIMAL(8,2) NOT NULL DEFAULT 0,
    daily_price DECIMAL(10,2) NOT NULL DEFAULT 0,
    monthly_price DECIMAL(10,2) NOT NULL DEFAULT 0,

```

```

available TINYINT(1) NOT NULL DEFAULT 1,
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
PRIMARY KEY (id),
KEY idx_properties_city (city),
KEY idx_properties_type (type),
KEY idx_properties_available (available)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-----
-- Seed: imoveis de exemplo
-----

INSERT INTO properties
(owner_id, title, description, type, city, address, bedrooms, bathrooms, area_m2, daily_price,
  ↳ monthly_price, available)
VALUES
(1, 'Apartamento Beira-Mar', 'Vista para o mar, mobiliado.', 'apartamento', 'Maceio', 'Av. Alvaro
  ↳ Otacilio, 1000', 2, 2, 78.50, 350.00, 4500.00, 1),
(1, 'Casa na Praia do Frances', 'Casa ampla com piscina.', 'casa', 'Marechal Deodoro', 'Rua das Conchas
  ↳ , 45', 4, 3, 220.00, 800.00, 9000.00, 1),
(1, 'Studio Centro', 'Compacto e bem localizado.', 'apartamento', 'Maceio', 'Rua do Comercio, 200', 1,
  ↳ 1, 35.00, 180.00, 2200.00, 1);

```

Listing 25: ms-catalogo/sql/db_catalogo.sql

A.4 Microservico ms-reservas e Aplicacoes

```

<?php

declare(strict_types=1);

/**
 * Front Controller do ms-reservas (porta 8002).
 *
 * O Gateway encaminha /api/reservas/* para ca, removendo o prefixo:
 * /api/reservas -> "/" (colecão)
 * /api/reservas/7 -> "/7" (item)
 */

use MsReservas\Config;
use MsReservas\Database;
use MsReservas\ReservationController;
use MsReservas\ReservationRepository;

$basePath = dirname(__DIR__);

$composerAutoload = $basePath . '/vendor/autoload.php';
if (is_file($composerAutoload)) {
    require $composerAutoload;
} else {
    spl_autoload_register(static function (string $class) use ($basePath): void {
        $prefix = 'MsReservas\\';
        if (!str_starts_with($class, $prefix)) {
            return;
        }
        $relative = str_replace('\\', '/', substr($class, strlen($prefix)));
        $file = $basePath . '/src/' . $relative . '.php';
        if (is_file($file)) {
            require $file;
        }
    });
}

Config::load($basePath);

$method = $_SERVER['REQUEST_METHOD'] ?? 'GET';
$path = '/' . trim(parse_url($_SERVER['REQUEST_URI'] ?? '/', PHP_URL_PATH) ?: '/', '/');

/** @return array<string,mixed> */
$readJsonBody = static function (): array {

```

```

    $raw = file_get_contents('php://input') ?: '';
    $data = json_decode($raw, true);
    return is_array($data) ? $data : [];
};

header('Content-Type: application/json; charset=utf-8');

$id = null;
if (preg_match('#~/(\\d+)#$', $path, $m)) {
    $id = (int) $m[1];
}

try {
    $controller = new ReservationController(new ReservationRepository(Database::connection()));

    match (true) {
        $method === 'GET' && $path === '/health' => print(json_encode(['service' => 'ms-reservas', 'status'
        => 'ok'])),
        $method === 'POST' && $path === '/' => $controller->store($readJsonBody()),
        $method === 'GET' && $path === '/' => $controller->index(),
        $method === 'GET' && $id !== null => $controller->show($id),
        default => (static function (): void {
            http_response_code(404);
            echo json_encode(['error' => 'Rota nao encontrada no ms-reservas.'], JSON_UNESCAPED_UNICODE);
        })(),
    };
} catch (\Throwable $e) {
    http_response_code(500);
    echo json_encode(['error' => 'Erro interno no ms-reservas.', 'detail' => $e->getMessage()],
        JSON_UNESCAPED_UNICODE);
}

```

Listing 26: ms-reservas/public/index.php

```

<?php

declare(strict_types=1);

namespace MsReservas;

use MsReservas\Engine\EngineFactory;
use MsReservas\Engine\ReservationException;
use MsReservas\Engine\ReservationRequest;

/**
 * Controlador de reservas.
 *
 * Recebe o payload, seleciona a modalidade via EngineFactory e delega o
 * processamento ao Template Method da RentEngine concreta.
 *
 * Endpoints (consumidos via Gateway em /api/reservas/):
 * POST / -> cria uma reserva (campo 'modality' define a engine)
 * GET / -> lista reservas
 * GET /{id} -> detalha uma reserva
 */
final class ReservationController
{
    public function __construct(private readonly ReservationRepository $repository)
    {
    }

    /**
     * @param array<string,mixed> $input
     */
    public function store(array $input): void
    {
        $modality = (string) ($input['modality'] ?? '');
        if ($modality === '') {
            $this->json(422, ['error' => "O campo 'modality' e obrigatorio."]);
            return;
        }

        try {

```

```

        $engine = EngineFactory::create($modality, $this->repository);
        $request = ReservationRequest::fromArray($input);
        $result = $engine->processReservation($request);

        $this->json(201, ['data' => $result->toArray()]);
    } catch (ReservationException $e) {
        $this->json($e->httpStatus(), ['error' => $e->getMessage()]);
    }
}

public function index(): void
{
    $this->json(200, [
        'data' => $this->repository->all(),
        'supported_modalities' => EngineFactory::supportedModalities(),
    ]);
}

public function show(int $id): void
{
    $reservation = $this->repository->find($id);
    if ($reservation === null) {
        $this->json(404, ['error' => 'Reserva nao encontrada.']);
        return;
    }
    $this->json(200, ['data' => $reservation]);
}

/**
 * @param array<string,mixed> $data
 */
private function json(int $status, array $data): void
{
    http_response_code($status);
    header('Content-Type: application/json; charset=utf-8');
    echo json_encode($data, JSON_UNESCAPED_UNICODE);
}
}

```

Listing 27: ms-reservas/src/ReservationController.php

```

<?php

declare(strict_types=1);

namespace MsReservas\Engine;

use MsReservas\ReservationRepository;

/**
 * RentEngine - Nucleo de reuso do framework (padrao Template Method, GoF).
 *
 * Define o ESQUELETO INVARIANTE do processo de reserva em
 * {@see RentEngine::processReservation()} (o "Template Method", marcado como
 * final para que subclasses nao alterem a ordem dos passos).
 *
 * Terminologia de frameworks:
 * - FROZEN SPOTS (pontos congelados): passos fixos, implementados aqui e
 * reutilizados por todas as modalidades - validacao base, verificacao de
 * disponibilidade e persistencia.
 * - HOT SPOTS (pontos quentes / pontos de variacao): passos que mudam por
 * modalidade - sao metodos abstratos (obrigatorios) ou hooks (opcionais).
 *
 * Hotspots desta engine:
 * - modalityName() [abstract] -> identifica a modalidade.
 * - calculatePrice() [abstract] -> regra de precificacao especifica.
 * - applyBusinessRules() [hook] -> regras extra (ex.: analise de credito).
 * - resolveStatus() [hook] -> status final da reserva.
 * - afterReservation() [hook] -> pos-processamento (ex.: notificacao).
 */
abstract class RentEngine
{
    public function __construct(protected readonly ReservationRepository $repository)

```



```

{
}

// =====
// TEMPLATE METHOD (frozen) - esqueleto invariante, nao sobrescrevivel.
// =====
final public function processReservation(ReservationRequest $request): ReservationResult
{
    // (1) FROZEN: validacao base, comum a todas as modalidades.
    $this->validateBaseRequest($request);

    // (2) FROZEN: verificacao de disponibilidade (conflito de datas).
    $this->ensureAvailability($request);

    // (3) HOTSPOT (abstract): cada modalidade calcula o preco a seu modo.
    $price = $this->calculatePrice($request);

    // (4) HOTSPOT (hook): regras de negocio adicionais e opcionais.
    $this->applyBusinessRules($request, $price);

    // (5) HOTSPOT (hook): define o status resultante (ex.: pending/confirmed).
    $status = $this->resolveStatus($request, $price);

    // (6) FROZEN: persistencia padronizada da reserva.
    $id = $this->repository->create(
        $request,
        $this->modalityName(),
        $price,
        $status,
        $this->reservationNotes($request)
    );

    // (7) HOTSPOT (hook): pos-processamento (ex.: enfileirar notificacao).
    $this->afterReservation($id, $request, $price);

    return new ReservationResult(
        id: $id,
        modality: $this->modalityName(),
        nights: $request->nights(),
        totalPrice: $price,
        status: $status,
        breakdown: $this->priceBreakdown($request, $price)
    );
}

// =====
// FROZEN SPOTS - passos fixos reutilizados por todas as modalidades.
// =====

private function validateBaseRequest(ReservationRequest $request): void
{
    if ($request->propertyId <= 0) {
        throw new ReservationException('Imovel (property_id) invalido.', 422);
    }
    if ($request->checkOut <= $request->checkIn) {
        throw new ReservationException('A data de check-out deve ser posterior ao check-in.', 422);
    }
    if ($request->guests < 1) {
        throw new ReservationException('Numero de hospedes invalido.', 422);
    }
}

private function ensureAvailability(ReservationRequest $request): void
{
    $conflict = $this->repository->hasConflict(
        $request->propertyId,
        $request->checkIn->format('Y-m-d'),
        $request->checkOut->format('Y-m-d')
    );

    if ($conflict) {
        throw new ReservationException('Imovel indisponivel para o periodo selecionado.', 409);
    }
}

```

```

}

// =====
// HOT SPOTS - pontos de variacao.
// =====

/** Identificador da modalidade (ex.: 'vacation', 'long_term'). */
abstract protected function modalityName(): string;

/** Regra de precificacao especifica da modalidade. */
abstract protected function calculatePrice(ReservationRequest $request): float;

/**
 * HOOK: regras de negocio adicionais. Implementacao padrao vazia
 * (modalidades simples nao precisam sobrescrever).
 */
protected function applyBusinessRules(ReservationRequest $request, float $price): void
{
    // noop por padrao.
}

/** HOOK: status final da reserva. Padrao: 'confirmed'. */
protected function resolveStatus(ReservationRequest $request, float $price): string
{
    return 'confirmed';
}

/** HOOK: pos-processamento apos persistir. Padrao: noop. */
protected function afterReservation(int $reservationId, ReservationRequest $request, float $price):
    ↪ void
{
    // noop por padrao.
}

/** HOOK: observacao gravada na reserva. Padrao: nenhuma. */
protected function reservationNotes(ReservationRequest $request): ?string
{
    return null;
}

/**
 * HOOK: detalhamento do preco para transparencia ao cliente.
 *
 * @return array<string,mixed>
 */
protected function priceBreakdown(ReservationRequest $request, float $price): array
{
    return [
        'modality' => $this->modalityName(),
        'nights' => $request->nights(),
        'total' => round($price, 2),
    ];
}
}

```

Listing 28: ms-reservas/src/Engine/RentEngine.php

```

<?php

declare(strict_types=1);

namespace MsReservas\Engine;

use MsReservas\Modalities\LongTermRent;
use MsReservas\Modalities\VacationRent;
use MsReservas\ReservationRepository;

/**
 * Fabrica de engines de reserva.
 *
 * Resolve a modalidade solicitada para a subclasse concreta de {@see RentEngine}.
 * Este e o ponto de extensao do framework: novas modalidades sao registradas
 * aqui sem alterar o nucleo (Open/Closed Principle).

```

```

*/
final class EngineFactory
{
    public static function create(string $modality, ReservationRepository $repository): RentEngine
    {
        return match ($modality) {
            'vacation' => new VacationRent($repository),
            'long_term' => new LongTermRent($repository),
            default => throw new ReservationException(
                "Modalidade de aluguel '{$modality}' nao suportada.",
                400
            ),
        };
    }

    /**
     * Modalidades atualmente disponiveis.
     *
     * @return array<int,string>
     */
    public static function supportedModalities(): array
    {
        return ['vacation', 'long_term'];
    }
}

```

Listing 29: ms-reservas/src/Engine/EngineFactory.php

```

<?php

declare(strict_types=1);

namespace MsReservas\Modalities;

use MsReservas\Engine\RentEngine;
use MsReservas\Engine\ReservationRequest;

/**
 * VacationRent - Aluguel por temporada.
 *
 * Demonstra o reuso por HERANCA: reaproveita integralmente o esqueleto do
 * Template Method (validacao, disponibilidade, persistencia) e sobrescreve
 * apenas os hotspots minimos obrigatorios. Por ser uma modalidade simples,
 * nao precisa dos hooks opcionais (usa o comportamento default da RentEngine).
 *
 * Regra de preco: diaria * numero de noites, com acrescimo de alta temporada
 * para reservas curtas (ate 3 noites) e taxa de limpeza fixa.
 */
final class VacationRent extends RentEngine
{
    private const CLEANING_FEE = 120.0;
    private const HIGH_SEASON_BONUS = 0.15; // 15% para estadias curtas

    protected function modalityName(): string
    {
        return 'vacation';
    }

    protected function calculatePrice(ReservationRequest $request): float
    {
        $nights = $request->nights();
        $subtotal = $nights * $request->dailyRate;

        if ($nights <= 3) {
            $subtotal *= (1 + self::HIGH_SEASON_BONUS);
        }

        return $subtotal + self::CLEANING_FEE;
    }

    /**
     * Detalhamento transparente do preco (sobrescreve o hook de breakdown).
     *

```

```

    * @return array<string,mixed>
    */
    protected function priceBreakdown(ReservationRequest $request, float $price): array
    {
        $nights = $request->nights();

        return [
            'modality' => $this->modalityName(),
            'nights' => $nights,
            'daily_rate' => round($request->dailyRate, 2),
            'cleaning_fee' => self::CLEANING_FEE,
            'high_season' => $nights <= 3,
            'total' => round($price, 2),
        ];
    }
}

```

Listing 30: ms-reservas/src/Modalities/VacationRent.php

```

<?php

declare(strict_types=1);

namespace MsReservas\Modalities;

use MsReservas\Components\CreditCheckComponent;
use MsReservas\Engine\RentEngine;
use MsReservas\Engine\ReservationException;
use MsReservas\Engine\ReservationRequest;
use MsReservas\ReservationRepository;

/**
 * LongTermRent - Aluguel de longa duracao (residencial).
 *
 * Combina duas tecnicas de reuso:
 * - HERANCA: reaproveita o Template Method da RentEngine (mesmo esqueleto
 * da VacationRent), sobrescrevendo o calculo de preco.
 * - COMPOSICAO: injeta o CreditCheckComponent e o aciona no hook
 * applyBusinessRules(), evidenciando que regras especificas entram por
 * pontos de variacao sem alterar o nucleo do framework.
 *
 * Regra de preco: valor mensal * numero de meses, com desconto progressivo
 * para contratos longos. A reserva so e confirmada se a analise de credito
 * for aprovada (caso contrario, fica 'pending' com a justificativa).
 */
final class LongTermRent extends RentEngine
{
    private const LONG_CONTRACT_MONTHS = 12;
    private const LONG_CONTRACT_DISCOUNT = 0.10; // 10% para contratos >= 12 meses

    /** @var array{approved:bool,required_income:float,informed_income:float,reason:string}|null */
    private ?array $creditResult = null;

    public function __construct(
        ReservationRepository $repository,
        private readonly CreditCheckComponent $creditCheck = new CreditCheckComponent()
    ) {
        parent::__construct($repository);
    }

    protected function modalityName(): string
    {
        return 'long_term';
    }

    protected function calculatePrice(ReservationRequest $request): float
    {
        $months = $request->months();
        $subtotal = $months * $request->monthlyRate;

        if ($months >= self::LONG_CONTRACT_MONTHS) {
            $subtotal *= (1 - self::LONG_CONTRACT_DISCOUNT);
        }
    }
}

```

```

        return $subtotal;
    }

    /**
     * HOOK sobrescrito: aciona o componente de analise de credito (composicao).
     */
    protected function applyBusinessRules(ReservationRequest $request, float $price): void
    {
        $this->creditResult = $this->creditCheck->evaluate($request);

        // Renda ausente/zerada e tratada como dado obrigatorio invalido.
        if ($this->creditResult['informed_income'] <= 0.0) {
            throw new ReservationException(
                'Informe a renda mensal (extras.monthly_income) para aluguel de longa duracao.',
                422
            );
        }
    }

    /**
     * HOOK sobrescrito: o resultado do credito define o status da reserva.
     */
    protected function resolveStatus(ReservationRequest $request, float $price): string
    {
        return ($this->creditResult['approved'] ?? false) ? 'confirmed' : 'pending';
    }

    /**
     * HOOK sobrescrito: registra a justificativa da analise de credito.
     */
    protected function reservationNotes(ReservationRequest $request): ?string
    {
        return $this->creditResult['reason'] ?? null;
    }

    /**
     * @return array<string,mixed>
     */
    protected function priceBreakdown(ReservationRequest $request, float $price): array
    {
        $months = $request->months();

        return [
            'modality' => $this->modalityName(),
            'months' => $months,
            'monthly_rate' => round($request->monthlyRate, 2),
            'long_contract' => $months >= self::LONG_CONTRACT_MONTHS,
            'credit_check' => $this->creditResult,
            'total' => round($price, 2),
        ];
    }
}

```

Listing 31: ms-reservas/src/Modalities/LongTermRent.php

```

<?php

declare(strict_types=1);

namespace MsReservas\Components;

use MsReservas\Engine\ReservationRequest;

/**
 * Componente reutilizavel de analise de credito.
 *
 * Exemplo de REUSO POR COMPOSICAO: em vez de embutir a regra em uma subclasse
 * via heranca, a funcionalidade e encapsulada aqui e "plugada" na modalidade
 * que precisar dela (no caso, LongTermRent). Outras modalidades poderiam
 * reaproveitar o mesmo componente sem qualquer relacao de heranca.
 */
final class CreditCheckComponent

```

```

{
    /**
     * @param float $minIncomeRatio Renda mensal minima exigida como multiplo
     * do valor mensal do aluguel (ex.: 3x).
     */
    public function __construct(private readonly float $minIncomeRatio = 3.0)
    {
    }

    /**
     * Avalia se a renda informada cobre o aluguel mensal com folga.
     *
     * @return array{approved:bool,required_income:float,informed_income:float,reason:string}
     */
    public function evaluate(ReservationRequest $request): array
    {
        $monthlyRent = $request->monthlyRate;
        $informedIncome = (float) $request->extra('monthly_income', 0.0);
        $requiredIncome = round($monthlyRent * $this->minIncomeRatio, 2);

        $approved = $informedIncome >= $requiredIncome;

        return [
            'approved' => $approved,
            'required_income' => $requiredIncome,
            'informed_income' => $informedIncome,
            'reason' => $approved
                ? 'Renda compativel com o aluguel.'
                : sprintf(
                    'Renda informada (R$ %.2f) inferior ao minimo exigido (R$ %.2f).',
                    $informedIncome,
                    $requiredIncome
                ),
        ];
    }
}

```

Listing 32: ms-reservas/src/Components/CreditCheckComponent.php

```

<?php

declare(strict_types=1);

namespace MsReservas;

use MsReservas\Engine\ReservationRequest;
use PDO;

/**
 * Acesso a dados da tabela reservations (db_reservas).
 */
final class ReservationRepository
{
    public function __construct(private readonly PDO $pdo)
    {
    }

    /**
     * Verifica se existe sobreposicao de datas para o mesmo imovel
     * (reservas nao canceladas/rejeitadas).
     */
    public function hasConflict(int $propertyId, string $checkIn, string $checkOut): bool
    {
        $stmt = $this->pdo->prepare(
            'SELECT 1 FROM reservations
            WHERE property_id = :pid
            AND status IN (\'pending\', \'confirmed\')
            AND check_in < :check_out
            AND check_out > :check_in
            LIMIT 1'
        );
        $stmt->execute([
            'pid' => $propertyId,

```

```

        'check_in' => $checkIn,
        'check_out' => $checkOut,
    ]);

    return (bool) $stmt->fetchColumn();
}

public function create(
    ReservationRequest $req,
    string $modality,
    float $totalPrice,
    string $status,
    ?string $notes = null
): int {
    $stmt = $this->pdo->prepare(
        'INSERT INTO reservations
        (property_id, user_id, modality, check_in, check_out, nights, total_price, status, notes)
        VALUES
        (:property_id, :user_id, :modality, :check_in, :check_out, :nights, :total_price, :status, :
        ↪ notes)'
    );
    $stmt->execute([
        'property_id' => $req->propertyId,
        'user_id' => $req->userId,
        'modality' => $modality,
        'check_in' => $req->checkIn->format('Y-m-d'),
        'check_out' => $req->checkOut->format('Y-m-d'),
        'nights' => $req->nights(),
        'total_price' => round($totalPrice, 2),
        'status' => $status,
        'notes' => $notes,
    ]);

    return (int) $this->pdo->lastInsertId();
}

/**
 * @return array<int,array<string,mixed>>
 */
public function all(): array
{
    return $this->pdo->query('SELECT * FROM reservations ORDER BY created_at DESC')->fetchAll();
}

/**
 * @return array<string,mixed>/null
 */
public function find(int $id): ?array
{
    $stmt = $this->pdo->prepare('SELECT * FROM reservations WHERE id = :id LIMIT 1');
    $stmt->execute(['id' => $id]);
    $row = $stmt->fetch();

    return $row ?: null;
}
}

```

Listing 33: ms-reservas/src/ReservationRepository.php

```

-- =====
-- Banco: db_reservas (Microserviço ms-reservas - porta 8002)
-- Engine do framework: persiste reservas geradas pelo Template Method.
-- Princípio: Database per Service (isolamento de dados).
-- =====

CREATE DATABASE IF NOT EXISTS db_reservas
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE db_reservas;

-- -----
-- Tabela de reservas

```

```
-- property_id e user_id referenciam logicamente outros bancos (sem FK fisica).
-- modality registra qual modalidade do framework gerou a reserva
-- (ex.: 'vacation', 'long_term'), evidenciando o reuso da RentEngine.
-----
CREATE TABLE IF NOT EXISTS reservations (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  property_id BIGINT UNSIGNED NOT NULL,
  user_id BIGINT UNSIGNED NULL,
  modality VARCHAR(40) NOT NULL,
  check_in DATE NOT NULL,
  check_out DATE NOT NULL,
  nights INT UNSIGNED NOT NULL DEFAULT 0,
  total_price DECIMAL(10,2) NOT NULL DEFAULT 0,
  status ENUM('pending','confirmed','rejected','cancelled') NOT NULL DEFAULT 'pending',
  notes VARCHAR(255) NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  KEY idx_reservations_property (property_id),
  KEY idx_reservations_modality (modality),
  KEY idx_reservations_dates (check_in, check_out)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Listing 34: ms-reservas/sql/db_reservas.sql

Referencias

Referências

- [1] GAMMA, E. et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [2] SOMMERVILLE, I. *Engenharia de Software*. 10. ed. Pearson, 2018.
- [3] NEWMAN, S. *Building Microservices*. 2. ed. O'Reilly, 2021.
- [4] SCHWABER, K.; SUTHERLAND, J. *The Scrum Guide*. 2020.
- [5] FOWLER, M. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [6] PHP DOCUMENTATION. *PHP 8.2 Manual*. Disponível em: <https://www.php.net/manual/>. Acesso em: 2026.